



ChatGPT API

RAG와 ChatGPT로 구축하는 AI응답시스템

필수 설치 및 API Key

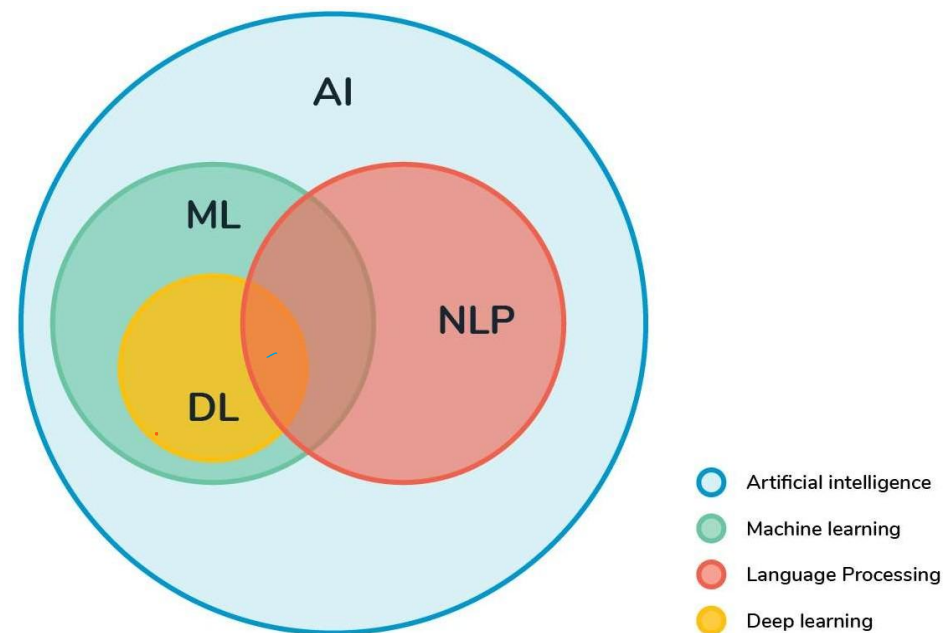
1. python 3.10.9
2. pip install openai
3. Visual studio code

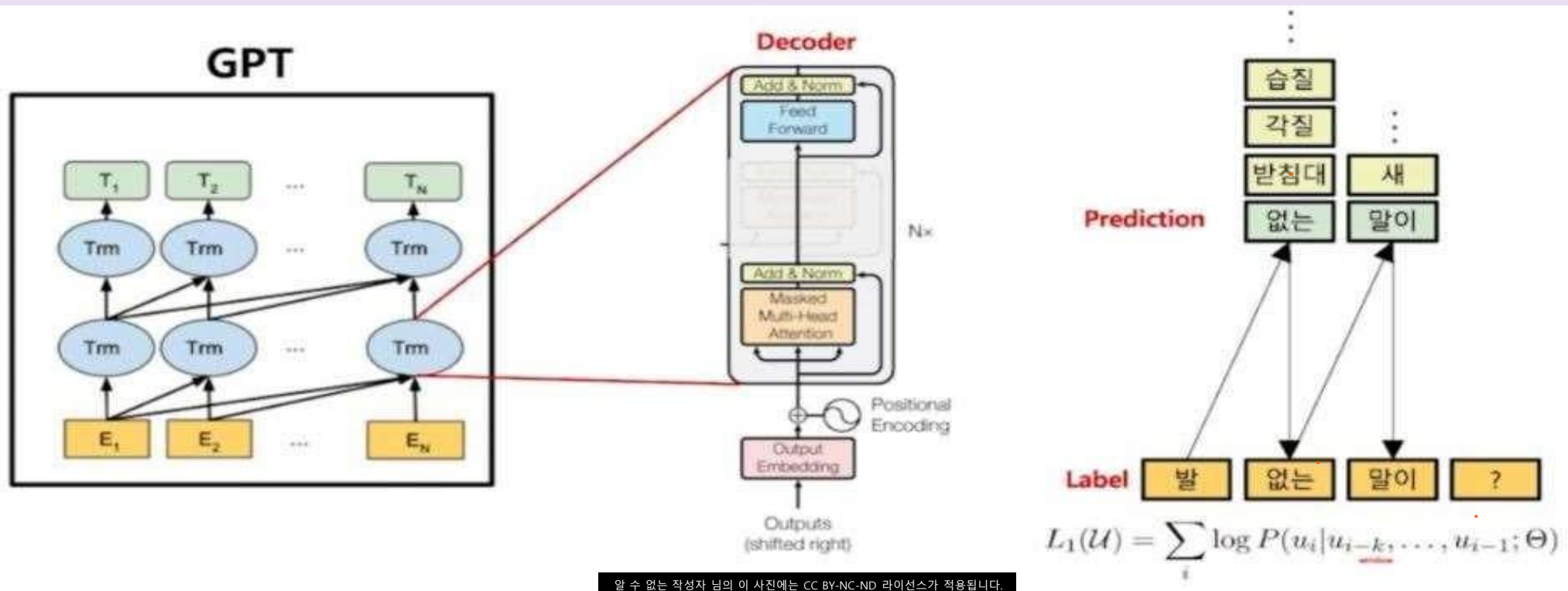
* GPT API Key

1. 자연어 처리 개요

자연어 처리와 인공지능의 관계

- 자연어 처리 기술의 학술적 위치
 - 철학: 언어철학
 - 과학(?): 언어학
 - 공학: 전산언어학 또는 응용언어학
 - 기술: 자연어처리
- 자연어 처리 기술과 인공지능 기술의 결합 역사
 - 규칙 기반 자연어 처리
 - 통계 중심 자연어 처리
 - 기계학습(머신러닝) 기술 중심 자연어 처리
 - 심층학습(딥러닝) 기술 중심 자연어 처리





자연어 생성 (NLG)

- 전통적인 처리 과정
 - 1. 생성할 내용 결정
 - 2. 메시지 구조화
 - 3. 메시지 표현 결정
- 최근의 처리 과정
 - 딥러닝 기반으로 위 세 단계를 한 번에 처리

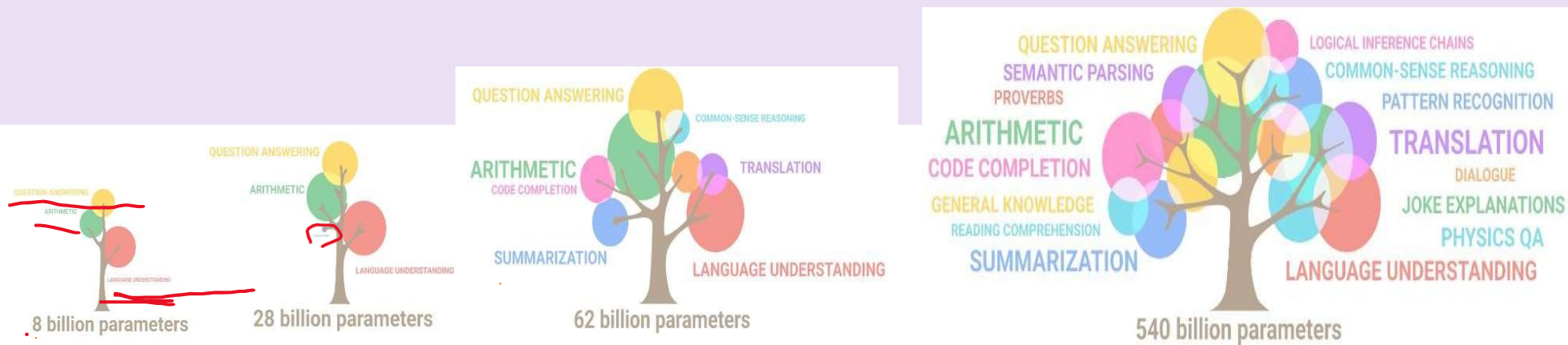


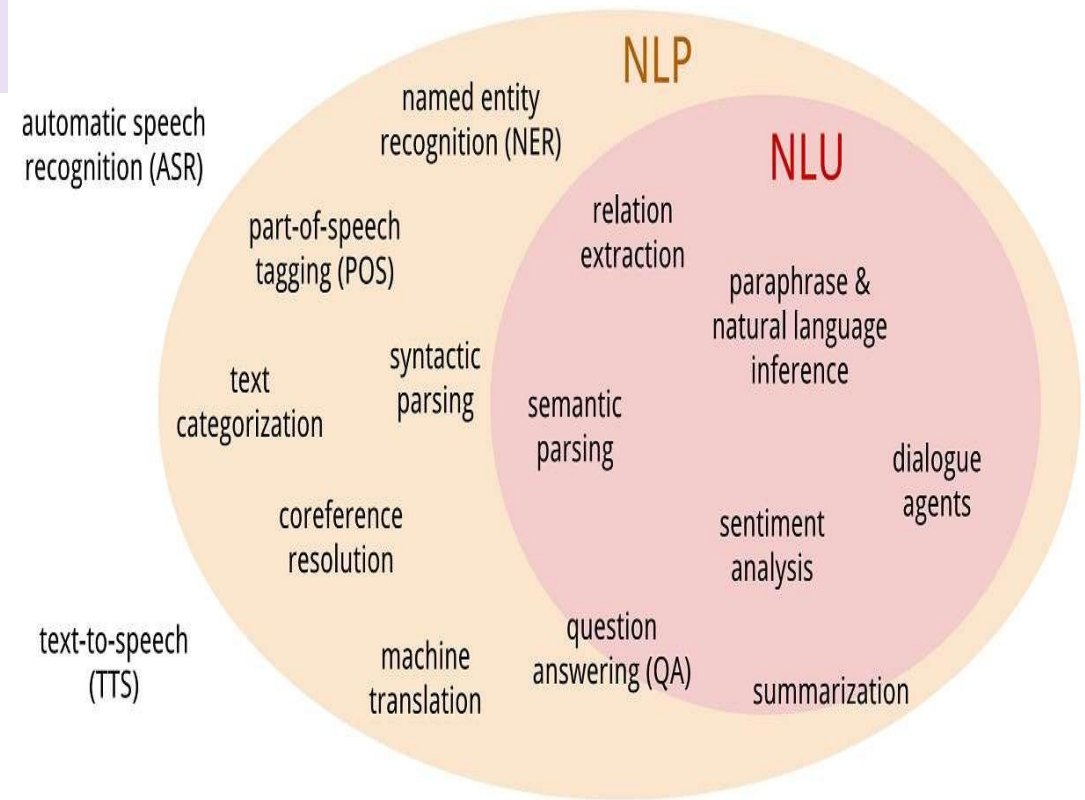
그림 출처: Shaian Naiang and Aakanksha Chowdhery, "Pathways Language Model (PaLM): Scaling to 540 Billion Parameters for Breakthrough Performance" 2022.4, Google Research(<https://ai.googleblog.com/2022/04/pathways-language-model-palm-scaling-to.html>)

자연어 처리 접근 방식

- 과거
 - 과업별 개별 기술 연구와 응용
 - 과업 해결에 규칙, 통계, 머신러닝, 딥러닝 방법을 동원해 봄
- 현재
 - 대규모 언어모델(LLM)으로 다양한 과업을 한 번에 해결 시도
 - 예: Chat-GPT로 분류, 분석, 번역, 질의응답 등이 모두 됨

자연어 처리

- 자연어
 - 인간이 일상적으로 사용하는 언어
- 자연어 처리
 - 인간의 일상 언어 처리
- 자연어 처리의 범주
 - 자연어 이해
 - 자연어 생성
- 자연어 처리 분야
 - 기계번역
 - 감정분석
 - 질의응답 시스템 등



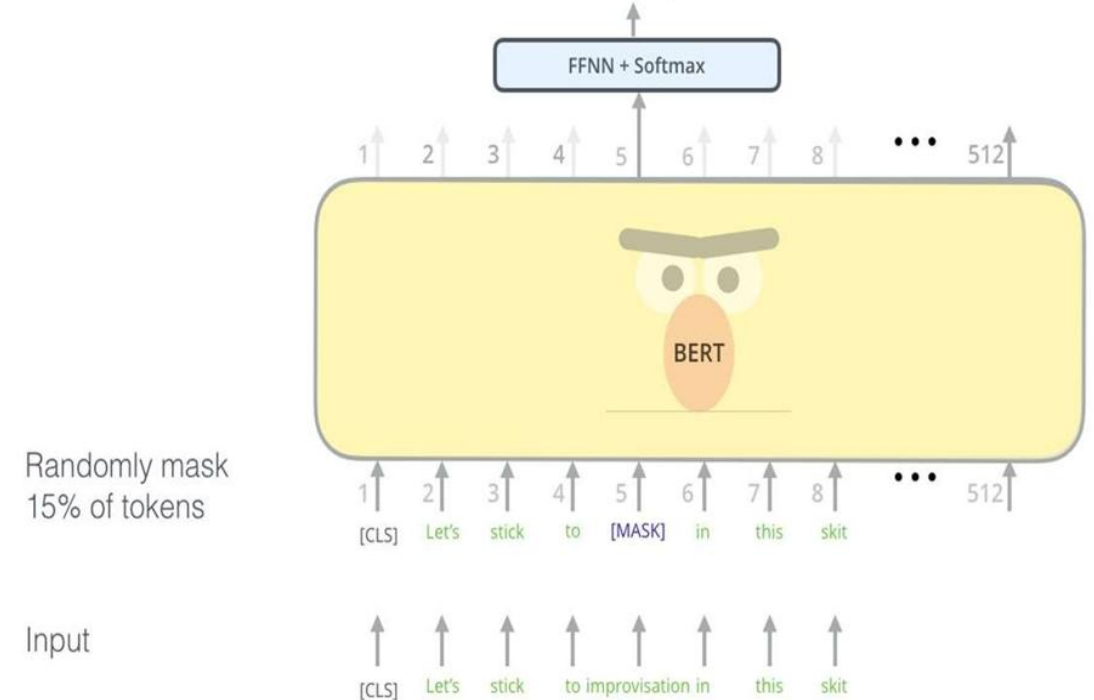
2. 언어 모형의 역사

언어모형(Language Models)

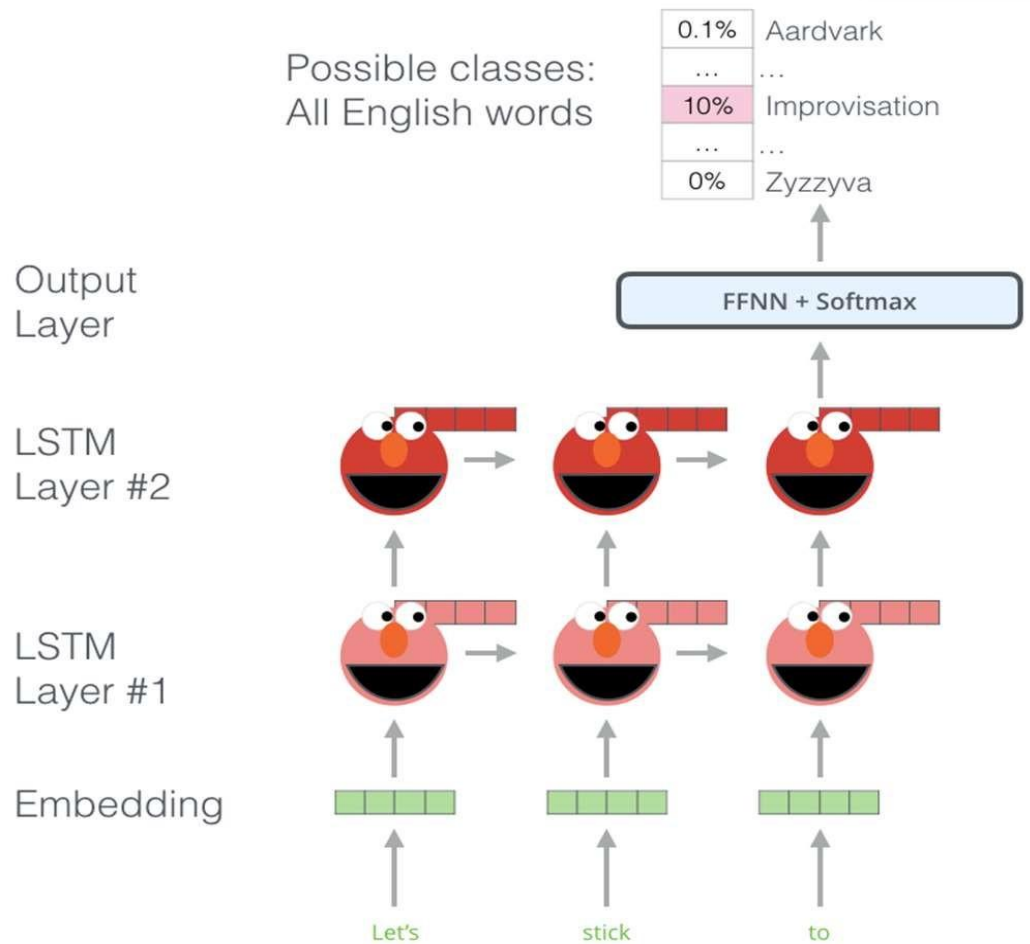
- 자연어 처리 기술의 핵심
- 일련의 단어가 출현(발생, 등장)할 확률을 추정하는 장치
- 언어모형을 반드시 인공신경망으로 만들어야만 하는 것은 아님
- 확률에 근거하여 문장을 생성하거나 번역을 수행할 수 있게 함

Use the output of the masked word's position to predict the masked word

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzyva



언어모형의 정의와 작동 원리



- 정의
 - 문장이나 단어의 등장 확률을 예측하는 모형
- 원리
 - 언어에는 순서가 존재
 - 이 순서에 근거해 다음에 출현할 단어가 무엇인지 예상 가능
 - 예상을 확률로 표현
 - 따라서 앞에 나온 단어들(즉, 문맥)이 무엇이나에 따라 확률이 달라짐

NLP 기술의 진화



n-gram: 최초의 (통계적) 언어모형

- 원리
 - 단어나 문자를 n 개씩 짝 지음
 - 예측할 단어보다 앞서 나온 $n-1$ 개 의 단어나 문자를 사용
 - 다음 단어나 문자의 등장 확률을 계산
- 장점
 - 간단한 모형
 - 계산 비용이 저렴
- 단점
 - n 이 커지면 불리(즉, 장기의존성이 약함)
 - 따라서 문맥(context) 파악에 불리
- 종류
 - 유니그램(uni-gram)
 - 바이그램(bi-gram)
 - 트라이그램(tri-gram: 트리그램)
 - 엔그램(n-gram)

This is Big Data AI Book

Uni-Gram

This	Is	Big	Data	AI	Book
------	----	-----	------	----	------

Bi-Gram

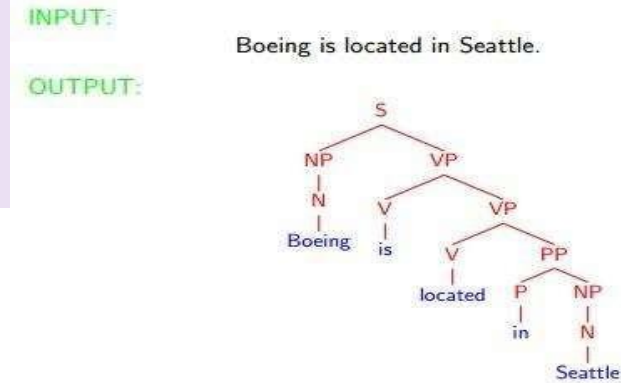
This is	Is Big	Big Data	Data AI	AI Book
---------	--------	----------	---------	---------

Tri-Gram

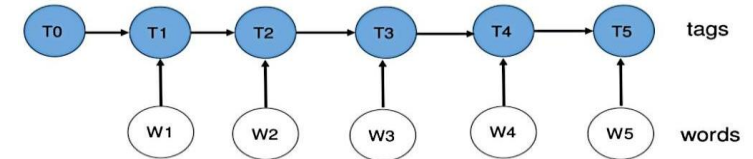
This is Big	Is Big Data	Big Data AI	Data AI Book
-------------	-------------	-------------	--------------

통계적 언어모형

- 원리
 - 텍스트 데이터의 통계적 패턴을 학습해 문장의 확률을 계산
- 장점
 - n-gram 모형보다 문맥을 더 잘 파악
- 단점
 - 여전히 장기 의존성을 고려하지 못함
- 종류
 - 은닉 마르코프 모형(Hidden Markov Model: HMM)
 - 잠재 디리클레 할당(Latent Dirichlet Allocation: LDA)
 - 확률적 문맥 자유 문법(Stochastic Context-Free Grammar: SCFG)
 - 기타



알 수 없는 작성자님의 사진에는 CC BY-SA 라이선스가 적용됩니다.

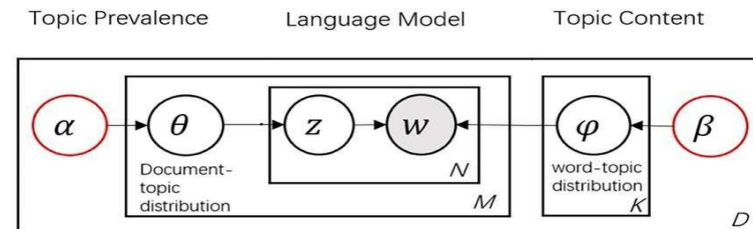


Discriminative model, model conditional probability $\Pr(T | W)$ directly.

$$\Pr(\mathbf{T} | \mathbf{W}) = \prod_{i=1}^L \Pr(t_i | t_{i-1}, w_i) = \prod_{i=1}^L \frac{\exp(\sum_j \beta_j f_j(t_{i-1}, w_i))}{Z(t_{i-1}, w_i)}$$

t_0 is a dummy start state.

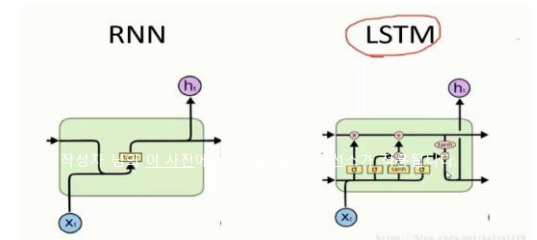
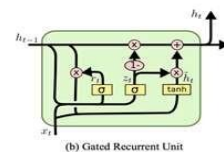
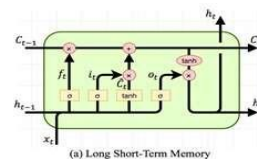
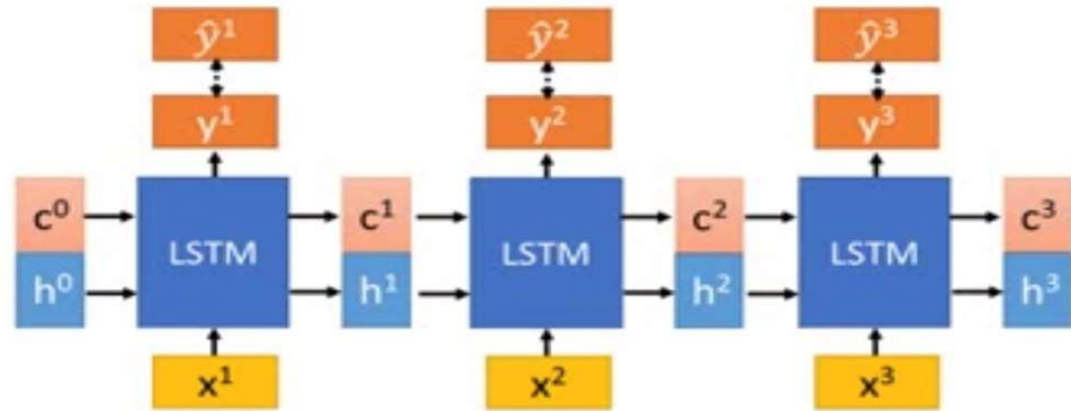
알 수 없는 작성자님의 사진에는 CC BY-SA 라이선스가 적용됩니다.



알 수 없는 작성자님의 사진에는 CC BY-SA 라이선스가 적용됩니다.

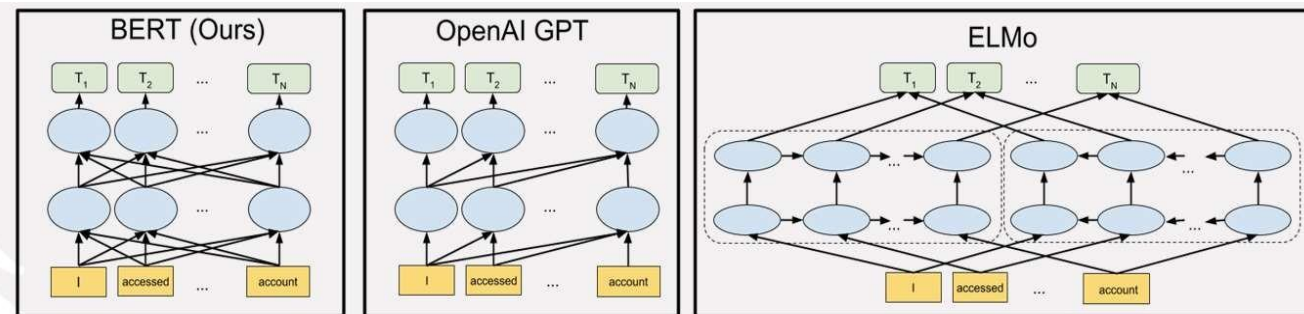
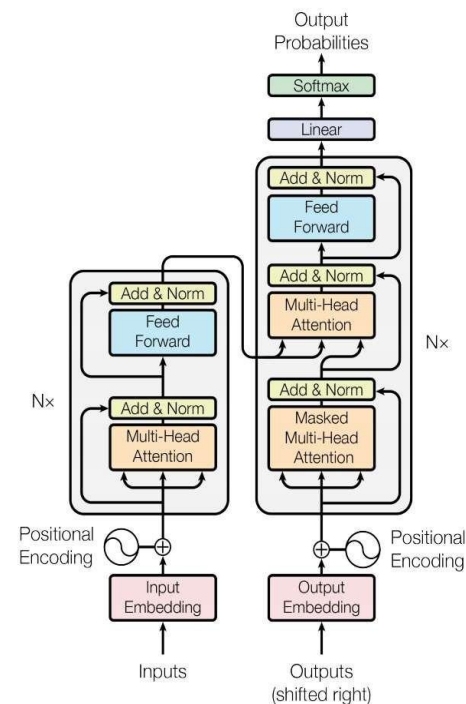
신경망 기반 언어 모형

- 원리
 - 신경망을 사용해 텍스트 데이터를 학습해 다음 단어나 문장의 출현 확률을 계산
- 장점
 - 장기적인 의존성을 고려
- 단점
 - 해석 불가능성
- 종류
 - RNN
 - LSTM
 - GRU



트랜스포머 구조 기반 언어모형

- 원리
 - 신경망 기반 언어모형에서 자기주의 기제 (self-attention mechanism)를 최대한 활용해 단어 간 관계를 파악
 - 이를 바탕으로 단어의 출현 확률을 더 정확 하게 계산
- 장점
 - 병렬 처리 가능
 - 장기 의존성 고려 가능
- 단점
 - 많은 메모리 소요
 - 해석 난해
- 종류
 - BERT
 - GPT
 - XLNet
 - 기타 다수



3.신경망기반 모델 구분

NLP

Language Model:

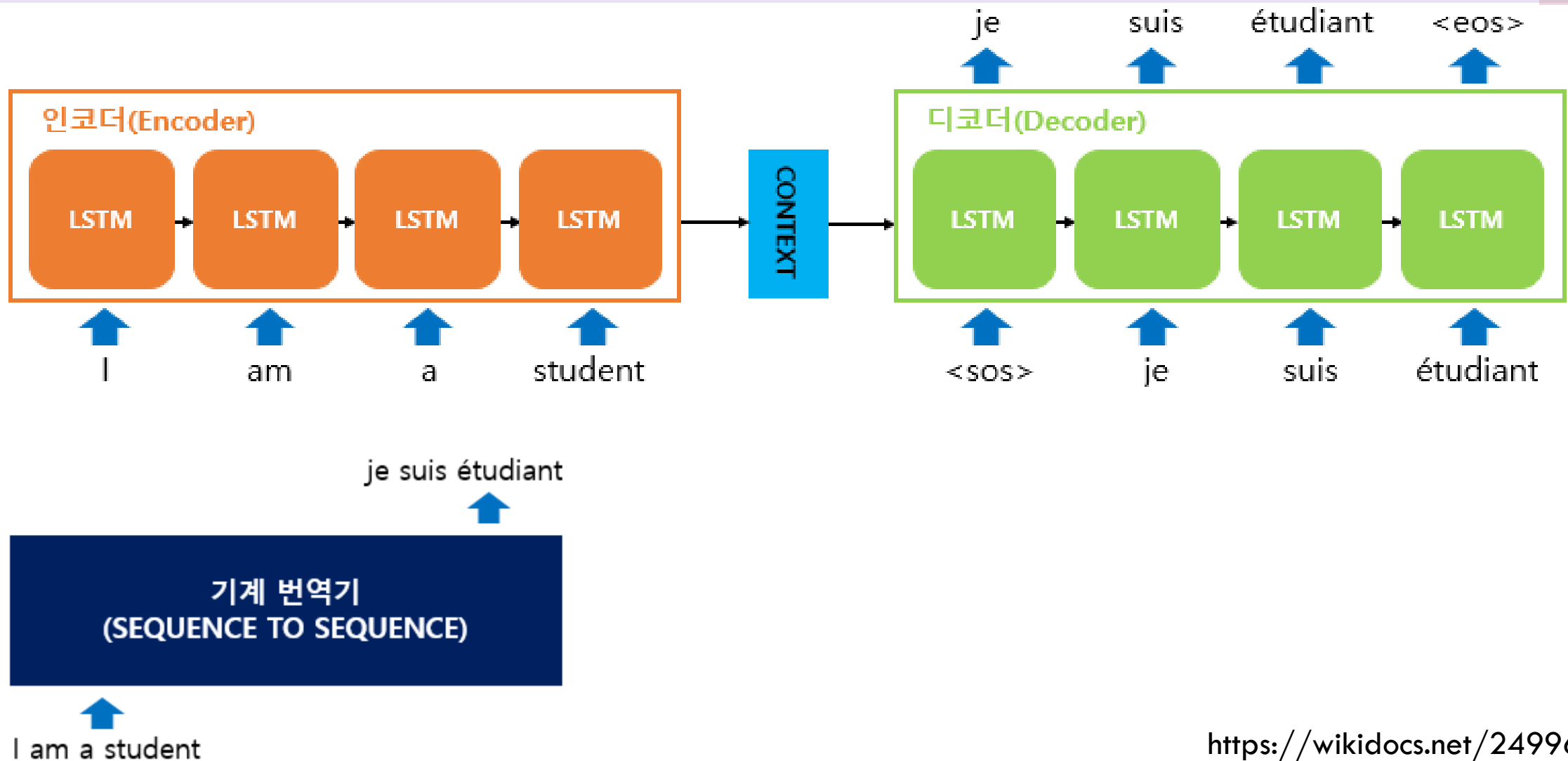
- 나열된 단어들의 조합에 확률을 부여하는 모델

Natural Language Understanding
(NLU)

Natural Language Generation
(NLG)

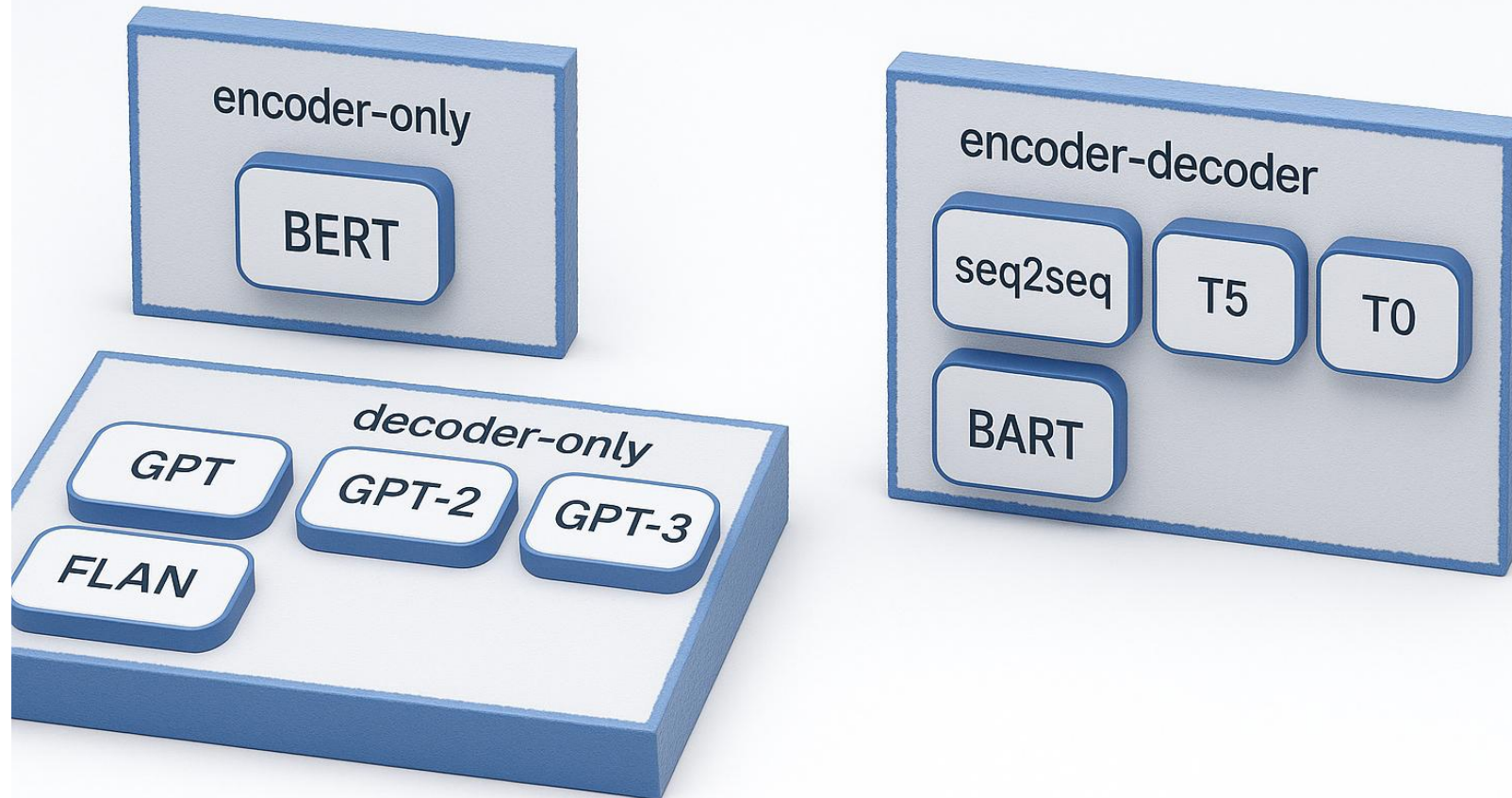


시퀀스-투-시퀀스(Sequence-to-Sequence):



encoder – decoder 에 따른 분류

models in this class: Architecture



사전학습 방식(pretrained)에 따른 분류

models in this class: Pre-training

seq2seq

NSP, MLM

denoising

LM

BERT

BART

GPT

T0

GPT-2

T5

GPT-3

FLAN

pre-trained

라벨이 없는 데이터를 학습

Self-supervised Learning:문제를 자동으로 생성해서 모델이 풀게함

항목	설명
☒ 학습 방식	자기지도 학습(Self-supervised Learning)
☒ 라벨 사용 여부	명시적 라벨 x (정답 레이블 없음)
☒ 예시	문장 중간 단어를 가리고(Masked Language Modeling), 다음 단어를 예측하거나 (Autoregressive Language Modeling) 문장 순서를 예측(NSP)

Self-supervised Learning

기존의 정답 레이블(label) 없이도,
입력 데이터로부터 스스로 학습 문제를 만들어 해결하는 방식.

예시:

BERT: 입력 문장에서 몇몇 단어를 [MASK]로 바꾸고, 그 단어가 무엇인지 맞힘 (MLM)

GPT: 문장을 앞에서부터 차례로 보고 다음 단어를 예측 (LM)

- LLM은 웹, 책, 논문, 위키피디아 등 **거대한 양의 텍스트 데이터**를 활용.
- 이 데이터 자체에서 학습 과제를 생성하여 학습하므로, 별도의 사람이 만든 라벨이 필요 없다.

사전학습방식 (NSP): 두개의 문장이 서로 이어지는지 여부

NLP 사전학습 방식 설명

1. NSP (Next Sentence Prediction)

- **사용 모델:** BERT
- **목표:** 두 문장이 문맥상 이어지는지 예측
- **예시:**
 - 문장 A: "나는 오늘 공원에 갔다."
 - 문장 B: "강아지가 뛰어다녔다."
 - 문장 B가 실제로 문장 A 뒤에 오는 문장인지
Yes/No 분류
- **용도:** 문장 간 관계 이해 (QA, NLI 등)

사전방습방식 (MLM): 텍스트 중간에 빈칸을 놓고 모델이 빈칸을 맞추는 방식

2. MLM (Masked Language Modeling)

- **사용 모델:** BERT
- **목표:** 문장에서 일부 단어를 [MASK] 처리한 후 이를 예측
- **예시:**
 - 입력: "나는 [MASK] 먹었다."
 - 출력: "사과"
- **장점:** 문맥을 양방향으로 고려하여 이해 가능
- **용도:** 텍스트 이해에 강점

Bert Mask



사전방습방식 (Denoising): 일부러 노이즈를 섞어서 원래로 교정하게 하는 방식

3. Denoising Autoencoding

- **사용 모델:** BART, T5
- **목표:** **손상된 입력(노이즈 추가)**을 원래 문장으로 복원
- **예시:**
 - 손상 입력: "나는 먹었다."
 - 정답: "나는 사과를 먹었다."
- **기법:**
 - 문장 순서 바꾸기
 - 단어 삭제/치환 등
- **특징:** 인코더-디코더 구조와 궁합이 좋음

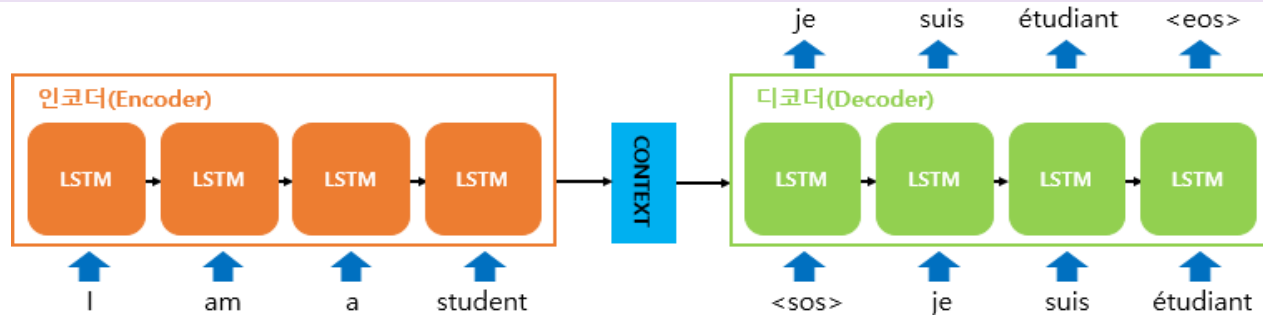
사전방습방식 (LM): 맥락이 주어졌을 때 다음 토큰을 예측하게 하는 방식

4. LM (Language Modeling)

- **사용 모델:** GPT, GPT-2, GPT-3, FLAN 등
- **목표:** 다음 단어 예측
- **예시:**
 - 입력: "나는 사과를"
 - 예측: "먹었다"
- **방식:** 왼쪽 → 오른쪽 순차적 예측 (auto-regressive)
- **용도:** 텍스트 생성에 탁월

사전학습 유형	설명	대표 모델	방향성
NSP	문장 연결 여부 예측	BERT	양방향
MLM	마스킹된 단어 예측	BERT	양방향
Denoising	손상된 문장을 원래대로 복원	BART, T5	인코더-디코더
LM (언어모델링)	다음 단어 예측	GPT 시리즈, FLAN	단방향 (좌→우)

Attention 등장 이유

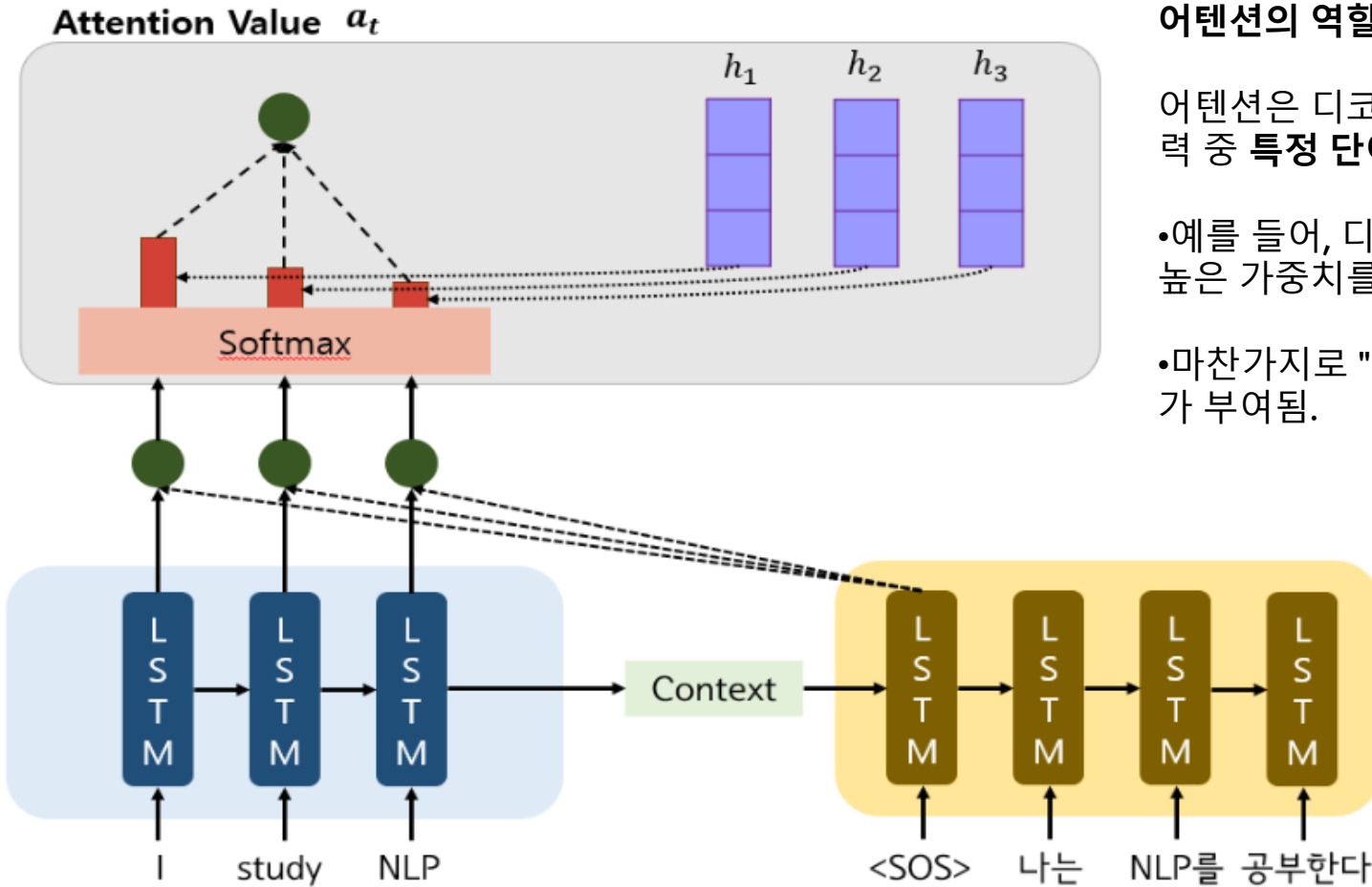


RNN에 기반한 seq2seq 모델에는 크게 두 가지 문제가 있다.

- 1) 하나의 고정된 크기의 벡터에 모든 정보를 압축하려고 하니까 정보 손실이 발생.
- 2) RNN의 고질적인 문제인 기울기 소실(vanishing gradient) 문제가 존재

결론: 입력 시퀀스가 길어지면 출력 시퀀스의 정확도가 떨어지는 것을 보정해주기 위한 등장한 기법

어텐션의 역할



어텐션의 역할

어텐션은 디코더가 한 번에 하나의 단어를 생성할 때, 인코더의 전체 출력 중 **특정 단어에 더 집중**할 수 있도록 돕는다.

•예를 들어, 디코더가 " 나는 " 을 생성하려고 할 때, 인코더의 " I " 에 높은 가중치를 부여.

•마찬가지로 "공부한다"를 생성할 때는 "study"와 "NLP"에 더 많은 가중치가 부여됨.

트랜스포머

- ▶ 각 토큰이 상호 중요도를 따져서 자연어를 처리 하겠다

Trainsformer의 주요 구성요소

Transformer의 주요 구성 요소와 Attention의 역할

1. Self-Attention:

- 입력 시퀀스의 각 토큰이 다른 모든 토큰들과 상호작용합니다. 이 과정을 통해 각 토큰이 전체 시퀀스에서 어떤 단어와 관계가 있는지를 파악하고, 중요한 단어에 더 높은 가중치를 부여합니다.

2. Multi-Head Attention:

- Self-Attention**을 여러 번 병렬로 실행하여 다양한 패턴을 학습합니다. 이를 통해 모델이 더 많은 정보를 한 번에 파악할 수 있으며, 더 복잡한 관계를 학습할 수 있습니다.

3. Positional Encoding:

- Transformer는 RNN과 달리 순차적인 구조가 없기 때문에, 입력 시퀀스의 단어 순서를 인식하기 위해 **Positional Encoding**이라는 위치 정보를 추가로 제공합니다. 이 위치 정보는 Attention 메커니즘에서 중요한 역할을 합니다.

4. Encoder-Decoder Attention (또는 Cross-Attention):

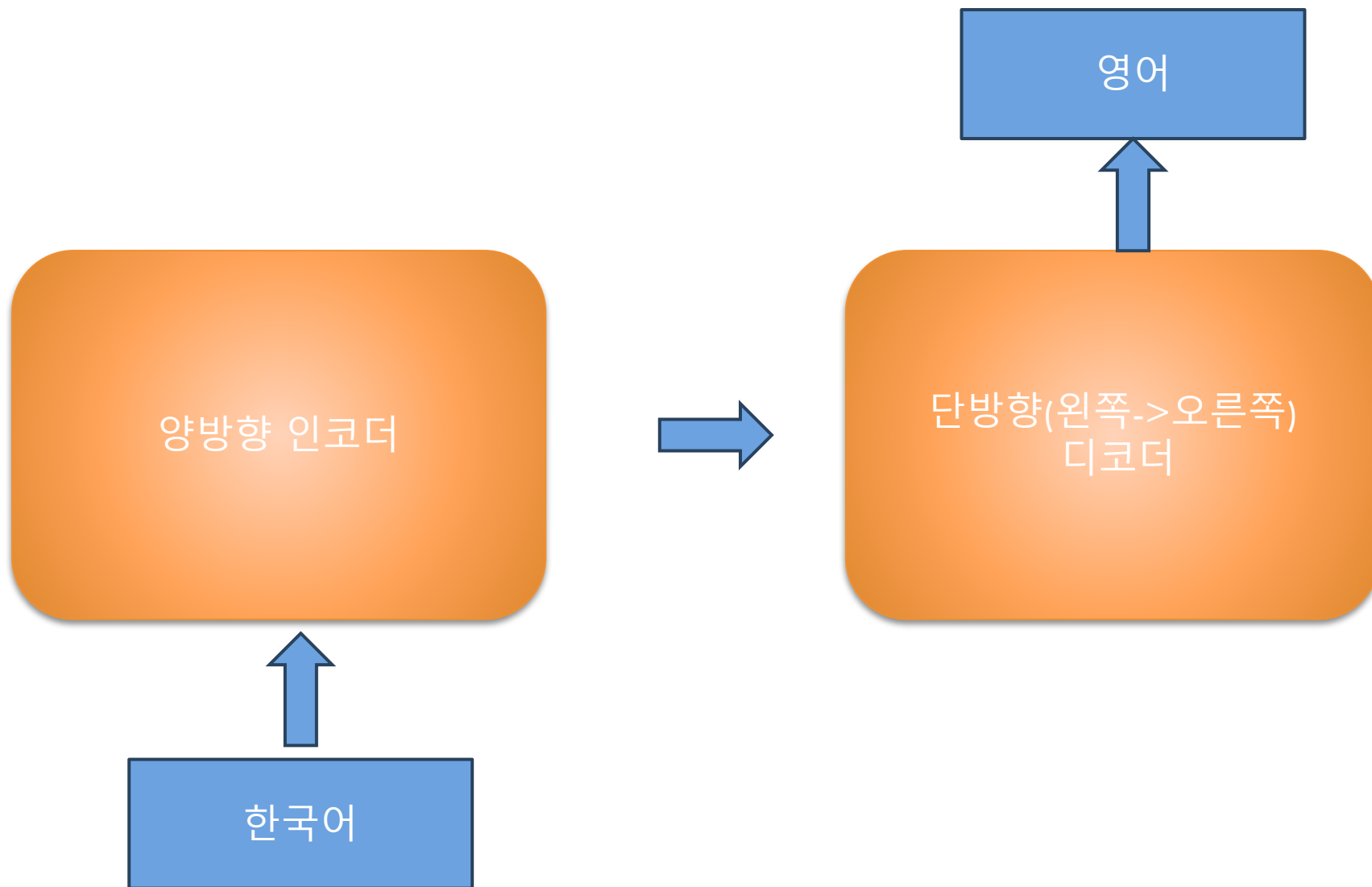
- 인코더에서 생성된 벡터와 디코더에서 생성된 벡터 간의 상호작용을 학습하기 위해 **Cross-Attention**을 사용합니다. 디코더는 인코더의 출력 정보를 활용하여 출력 시퀀스를 생성합니다.

3.버트 모델

Bert

- ▶ **BERT**(Bidirectional Encoder Representations from Transformers)는 2018년 구글에서 발표한 **Transformer 기반의 사전 학습된 언어 모델**.
- ▶ BERT는 자연어 처리(NLP) 작업에서 혁신적인 성과를 보였으며, 다양한 언어 이해 작업에서 뛰어난 성능을 보임

버트모델



Bert Mask



BERT의 주요 특징1

1. Bidirectional(양방향):

- 1) BERT는 텍스트의 양방향 컨텍스트를 이해할 수 있다. 즉, 단어의 좌우 맥락을 동시에 고려하여 더 깊은 의미를 학습
- 2) 예전의 Transformer 모델들은 주로 한 방향(왼쪽에서 오른쪽, 또는 오른쪽에서 왼쪽)으로만 학습했으나, BERT는 양방향으로 학습하여 단어의 문맥을 더 잘 이해할 수 있다.

2. Transformer 인코더 구조:

- 1) BERT는 Transformer의 인코더(Encoder) 부분만을 사용합니다. 디코더는 사용하지 않습니다.
- 2) Transformer 인코더는 Self-Attention 메커니즘을 사용해 입력 시퀀스 내에서 모든 단어가 서로 어떤 관계가 있는지를 학습.

BERT의 주요 특징2

3. 사전 훈련 (Pre-training):

1) BERT는 두 가지 주요 목표로 사전 훈련.

1-1. Masked Language Model (MLM): 입력 문장의 일부 단어를 마스킹(masking)하여, 모델이 그 단어를 예측하도록 한다. 예를 들어, 문장에서 일부 단어를 가리고, 나머지 단어들을 기반으로 그 단어를 예측하게 한다.

1-2. Next Sentence Prediction (NSP): 두 개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 나올 가능성이 높은지를 예측합니다. 이를 통해 문장 간의 관계를 학습.

4. 미세 조정 (Fine-tuning):

- 1) BERT는 사전 훈련된 모델을 특정한 NLP 작업(문장 분류, 문장 관계 예측, 질의응답 등)에 맞게 미세 조정(fine-tuning)하여 성능을 최적화할 수 있다.
- 2) 소량의 태스크 전용 데이터만으로도 높은 성능을 달성할 수 있다.

4. LLM(LLM) (Large Language Models)

LLM의 정의

- 대규모 언어 데이터를 기반으로 사전학습이 된 언어모형

LLM의 특징

- 언어의 다양한 특성을 학습
- 이로 인해 다양한 NLP 과업에 범용 적으로 사용 가능
- 최근에는 트랜스포머 구조를 많이 사용

LLM 이란

AI :컴퓨터 사람을 모방 하는 모든 기술

머신러닝 :학습을 통한 모델추론

딥러닝 : 인공신경망 적용

Gen AI :생성형 AI

LLM :텍스트

LLM vs GAI

- LLM (대규모 언어 모델):

- 대량의 텍스트 데이터를 학습하여 자연어 처리(NLP) 작업을 수행하는 AI 모델입니다.
- 주로 텍스트 생성, 번역, 요약, 질의응답 등 언어와 관련된 작업에 사용됩니다.
- 예: ChatGPT, GPT-4, LLaMA.

- GAI (생성적 인공지능, Generative AI):

- 텍스트뿐만 아니라 이미지, 음악, 코드 등 다양한 유형의 콘텐츠를 생성하는 AI 기술입니다.
- LLM은 GAI의 한 부분이며, GAI는 언어뿐만 아니라 이미지 생성(AI 이미지 생성 도구), 음악 작곡, 영상 생성 등의 다양한 창작 활동을 포함합니다.
- 예: DALL-E (이미지 생성), MidJourney (이미지 생성), Stable Diffusion (이미지 생성).

차이:

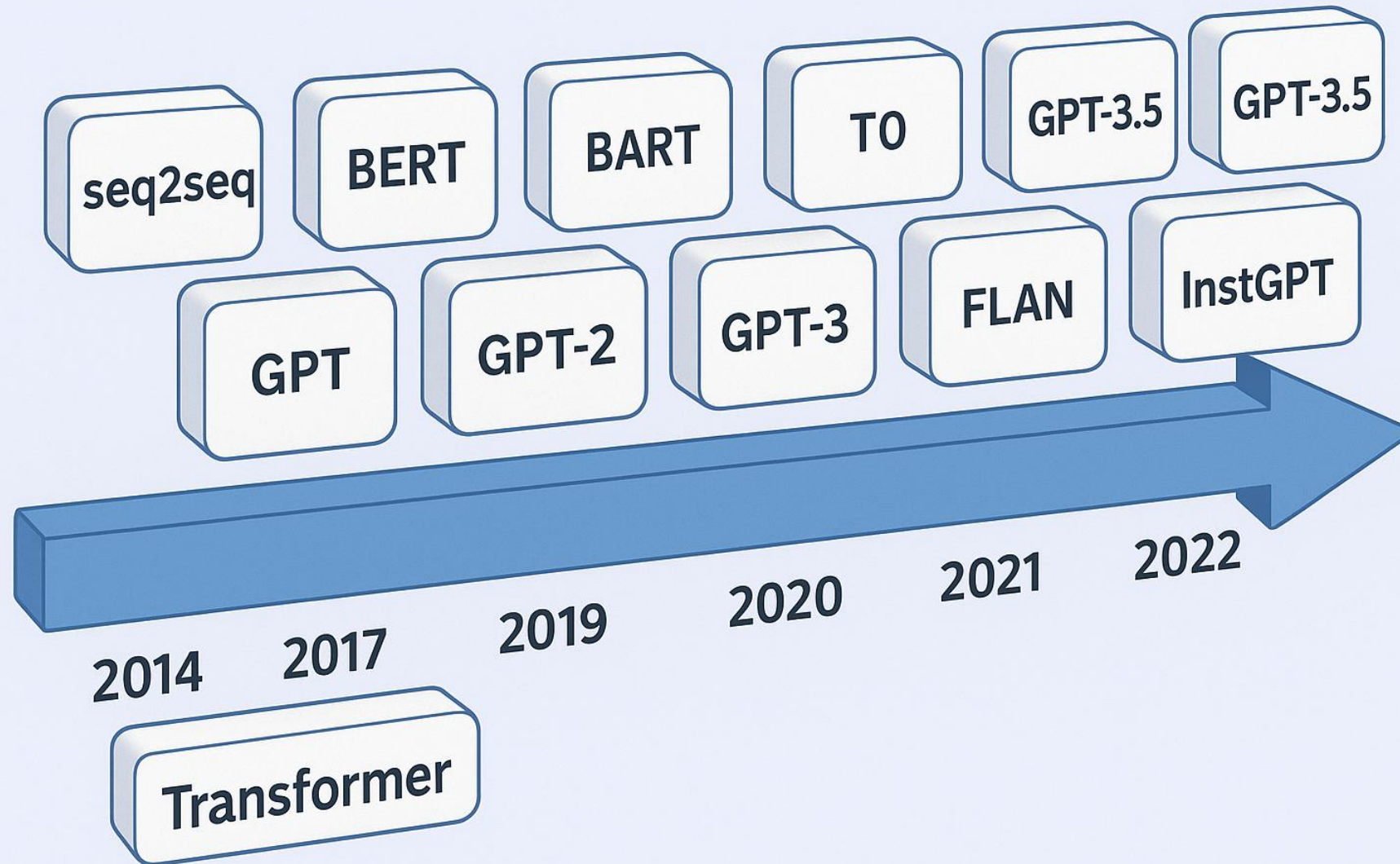
- LLM은 주로 언어 모델에 초점을 맞추고 있고,
- GAI는 다양한 형태의 데이터를 생성하는 포괄적인 AI 기술입니다.

SLM vs LLM

특성	SLM (Small Language Model)	LLM (Large Language Model)
모델 크기	상대적으로 작은 크기 (수백만에서 수십억 개의 파라미터)	매우 큰 크기 (수십억에서 수조 개의 파라미터)
학습 데이터	제한된 양의 데이터로 학습	방대한 양의 데이터로 학습
연산 자원	상대적으로 적은 연산 자원 사용	높은 연산 자원 필요 (고성능 GPU/TPU 필요)
속도	빠른 응답 속도	느린 응답 속도 (모델 크기와 연산량에 따라 달라짐)
정확도	제한적 정확도, 주로 단순한 작업에 적합	높은 정확도, 복잡한 언어 작업 수행 가능
용도	특정 작업 또는 리소스가 제한된 환경에서 사용	다양한 고급 언어 처리 작업 (대화형 AI, 텍스트 생성 등)
모델 훈련 시간	짧은 시간 안에 훈련 가능	긴 훈련 시간 필요 (수주~수개월 소요)
예시	간단한 문서 분류기, 간단한 텍스트 생성 작업	ChatGPT, GPT-4, Google Gemini, Meta LLaMA 등
응용 분야	기본적인 NLP 태스크(문서 분류, 감정 분석 등)	고급 NLP 태스크 (대화형 AI, 코드 생성, 문서 요약 등)

5.11m 모델역사

2019년부터 2023년까지의 LLM 발전사를 보면. 각 모델은 출시 시기와 기술적 진보에 따라 연결되어 있으며, **구글(Google), OpenAI, Meta(구 Facebook), Microsoft, Anthropic** 등 다양한 기업이 언어 모델 개발에 참여하고 있음



모델 역사

1. 2019년:

- **GPT-3**: OpenAI의 GPT-3 모델이 등장하며 대규모 언어 모델의 시대가 본격적으로 시작됩니다. 이는 이후 다양한 모델 발전의 기초가 됩니다.

2. 2020년:

- 구글의 **T5**, **mT5**, **GShard** 등 여러 모델이 발표되며 다국어 및 대규모 데이터 처리 능력이 강화됩니다.
- **Codex**(OpenAI)와 **PanGu- α** (화웨이), **PLUG**(Alibaba)와 같은 모델들도 이 시기에 나왔습니다.

3. 2021년:

- 구글의 **LaMDA**, **FLAN** 등 고도화된 언어 모델이 등장합니다.
- **HyperCLOVA**(NAVER), **Ernie 3.0**(Titan), **Gopher**(DeepMind) 등의 모델들이 발표되어 다양한 언어와 기능적 진보가 이루어집니다.
- 또한 Meta의 **OPT** 및 **Galactica**, **BLOOM**, **CodeGen** 등 다양한 모델이 출시됩니다.

4. 2022년:

- 구글의 **PaLM**, **Chinchilla**, **AlphaCode**, **LaMDA**, **UL2** 등이 발표되며, 이 시기는 언어 모델의 활용도와 적용 범위가 넓어지는 시기입니다.
- OpenAI는 **InstructGPT**를 통해 더욱 사용자 중심적인 모델을 선보였으며, **Anthropic**도 활발히 참여합니다.
- Meta는 **LLaMA**, **OPT** 등의 모델을 통해 언어 모델 생태계를 주도합니다.

모델 역사

5. 2023년:

- OpenAI는 **ChatGPT**와 **GPT-4**를 발표하며 LLM의 대중화에 큰 기여를 합니다.
- Meta는 **LLaMA2**를 발표하며 LLM의 성능과 활용성을 크게 발전시킵니다.
- 구글은 **PaLM2**, **Bard**, **Flan-PaLM** 등의 모델을 출시하며 언어 모델 기술을 선도합니다.
- 이 외에도 **Falcon**, **InternLM**, **Vicuna** 등 다양한 모델들이 등장하여 LLM 기술이 빠르게 확산되고 있습니다.

특징:

- 연구의 폭과 깊이가 지속적으로 증가하고 있으며, 언어 모델은 더욱 정교해지고 있습니다.
- 각 기업은 다양한 모델을 출시하면서 자연어 처리, 코딩, 의사소통, 번역 등 여러 응용 분야에 모델을 적용하고 있습니다.
- 2023년을 기준으로, **GPT-4**, **LLaMA2** 등의 최신 모델들이 많은 주목을 받고 있으며, 이들 모델은 추론 능력, 대화 처리, 복잡한 질문에 대한 답변 능력에서 큰 성과를 내고 있습니다.

국내 AI 모델

모델명	개발사/기관	특징
KoGPT	카카오브레인	한국어에 특화된 GPT 모델로, 대용량 한국어 데이터를 학습하여 자연스러운 한국어 생성 가능
KorBERT	SK텔레콤	BERT 기반의 한국어 언어 모델로, 다양한 자연어 처리 태스크에 활용 가능
HanBERT	한양대학교	한국어 문장 임베딩을 위한 BERT 모델로, 문장 유사도 분석 등에 사용
ETRI AI 모델	한국전자통신연구원	다양한 한국어 처리 태스크를 위한 언어 모델을 개발하여 산업계에 제공
NAVER HyperCLOVA	네이버	대용량 한국어 데이터를 학습한 초대형 언어 모델로, 다양한 언어 이해 및 생성 작업 수행

중국 AI 모델

1. 딥시크(DeepSeek) R1

중국의 AI 스타트업 딥시크(DeepSeek)가 2025년 1월 20일에 출시한 추론 특화 AI 모델입니다. 딥시크 R1은 미국 수학경시대회 벤치마크 테스트에서 우수한 성능을 보였으며, 개발비용이 상대적으로 저렴하고 개발 기간도 짧아 실리콘밸리에 충격을 주었습니다. [HANKOOKILBO.COM](#)

2. 알리바바의 큐원(Qwen) 2.5-맥스

알리바바 클라우드는 2025년 1월 29일에 새로운 AI 모델인 '큐원(Qwen) 2.5-맥스'를 출시했습니다. 이 모델은 GPT, 딥시크, 라마 등의 모델을 능가하는 성능을 갖추었다고 주장하고 있습니다. [KOREATIMES.COM](#)

3. 문샷(Moonshot) AI의 LLM

문샷AI는 2025년 1월 20일에 대규모 언어 모델(LLM)을 출시했습니다. 이 모델은 수학과 추론 분야에서 오픈AI의 모델에 도전할 수 있는 성능을 갖추었다고 주장하고 있습니다. [JIMAGAZINEJOINS.COM](#)

4. 바이두(Baidu)의 어니(ERNIE) X1 및 어니 4.5

바이두는 2025년 3월 16일에 두 가지 새로운 AI 모델을 발표했습니다. 어니 X1은 추론 중심 모델로, DeepSeek의 모델과 유사한 성능을 절반의 비용으로 제공하며, 이해, 계획, 반성, 진화 능력이 향상되었습니다. 어니 4.5는 멀티모달 이해와 고급 언어 능력을 갖춘 모델로, 이해, 생성, 논리 및 기억 능력이 향상되었으며, 높은 감성 지수를 보유하고 있습니다. [REUTERS.COM](#)

5. 바이트댄스(ByteDance)의 AI 모델

틱톡 운영사인 바이트댄스도 최근 AI 모델 업데이트를 발표하며, 새 모델의 성능이 오픈AI 모델을 뛰어넘는다고 주장하고 있습니다. [KHAN.CO.KR](#)

이러한 모델들의 등장은 중국이 AI 분야에서 빠르게 발전하고 있음을 보여주며, 글로벌 AI 경쟁에서 주목받고 있습니다.

6. LLM 계층별 모델 종류

- **Application Layer:** LLM을 활용한 실질적인 서비스 (카피라이팅, 코딩, 챗봇 등).
- **Infrastructure Layer:** LLM을 개발하고 실행하는 데 필요한 모델, 하드웨어, 미세 조정, 추론 기술 제공.

Large Language Models

BCV

Application Layer

계층별 모델 종류

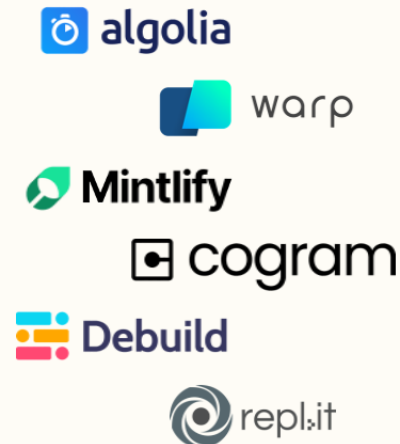
Copywriting



Coding



Dev Tools



Chat / Comms

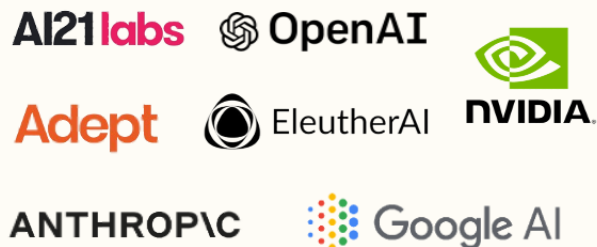


BizOps



Infrastructure Layer

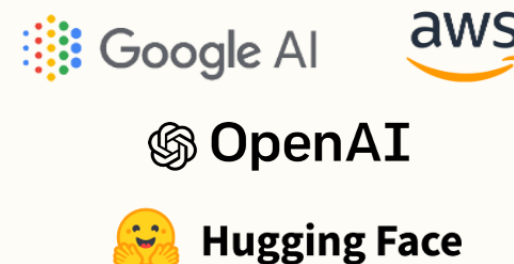
Model Creation



Hardware



Fine Tuning



Inference



인프라 계층

인프라 계층:

이 계층은 LLM 개발과 확장을 지원하는 기초 기술과 서비스에 중점을 둡니다.

1. 모델 생성(Model Creation):

- **AI21 Labs, OpenAI, Google AI, Anthropic, NVIDIA** 등의 기업은 직접 대규모 언어 모델을 개발하고 관련 기술을 발전시키는 데 참여하고 있습니다.

2. 하드웨어(Hardware):

- **Habana, NVIDIA, Cerebras, SambaNova Systems, Graphcore** 등의 하드웨어 제공업체는 대규모 모델을 학습시키는 데 필요한 계산 인프라를 제공합니다.

3. 모델 미세 조정(Fine Tuning):

- **Google AI, AWS, OpenAI, Hugging Face** 등은 사전 학습된 모델을 특정 작업이나 맞춤형 데이터셋에 맞게 미세 조정할 수 있는 플랫폼을 제공합니다.

4. 추론(Inference):

- **OpenAI와 Hugging Face**는 훈련된 모델을 실시간으로 사용할 수 있도록 추론 서비스를 제공하며, 이를 통해 챗봇, 자동 고객 지원 등의 실시간 애플리케이션을 지원합니다.

애플리케이션 계층

애플리케이션 계층:

이 계층은 LLM 기술을 활용하여 특정 기능과 사용 사례를 제공하는 도구와 서비스들로 구성되어 있습니다.

1. 카피라이팅(Copywriting):

- **Jasper, copy.ai, HyperWrite, Headline** 등의 도구는 대규모 언어 모델을 사용해 블로그 글, 마케팅 카피 등의 콘텐츠를 자동 생성하거나 작성 보조 기능을 제공합니다.

2. 코딩(Coding):

- **Tabnine, GitHub Copilot, Codiga, Mutable AI** 등의 플랫폼은 AI 기반 코딩 보조 기능을 제공하여, 실시간 코드 제안, 자동 완성 및 전체 코드 구현을 도와줍니다.

3. 개발 도구(Dev Tools):

- **algolia, warp, Mintlify, cogram, Debuild, replit** 같은 도구들은 AI를 통해 개발자의 작업 흐름을 개선하고 자동화를 지원합니다.

4. 채팅/통신(Chat/Comms):

- **MessageBird, Replier.ai, Sapling, Fable** 등의 애플리케이션은 AI 모델을 사용해 고객 지원 및 통신 기능을 최적화하고 자동화된 채팅 기능을 제공합니다.

5. 비즈니스 운영(BizOps):

- **Viable, Interpret, Tabulate, Anecdote, CaseText** 같은 솔루션은 대규모 언어 모델을 사용해 비즈니스 운영, 데이터 분석 및 콘텐츠 이해를 향상시킵니다.

애플리케이션 계층

◆ Application Layer (응용 계층)

LLM을 활용하여 특정 기능을 제공하는 서비스들입니다.

1. 📄 Copywriting (카피라이팅)

- **Jasper, Copy.ai, Writesonic, Cohere** 등
- AI 기반의 콘텐츠 생성 도구로, 블로그 글, 광고 문구, 이메일, 소셜 미디어 콘텐츠를 자동 생성함.
- 마케팅, 전자상거래, 블로그 등에 활용됨.

2. 💻 Coding (코딩)

- **GitHub Copilot, Tabnine, Mutable AI, Codiga, Cohere, Codegen** 등
- AI를 활용한 코드 자동 완성 및 코드 최적화 지원.
- 개발자의 생산성을 높여 코드 작성 속도를 향상시키는 역할.

3. 🛠️ Dev Tools (개발 도구)

- **Algolia, Warp, Mintlify, Cogram, Debuild, Replit**
- LLM을 활용한 검색 기능 향상(Algolia), 코드 협업 및 배포 자동화(Warp, Replit) 등의 도구.

4. 💬 Chat / Comms (채팅 및 커뮤니케이션)

- **MessageBird, Replier.ai, Sapling, Fable**
- 고객 지원 및 비즈니스용 AI 챗봇 서비스 제공.
- 이메일 자동 응답, 고객 문의 대응, 대화형 AI 기반 고객 서비스 강화.

5. 📊 BizOps (비즈니스 운영)

- **Viable, Enterpret, Tabulate, Anecdote, Otherside AI, Casetext, Dover**
- AI 기반으로 비즈니스 운영을 최적화하는 서비스.
- 예를 들어, 고객 피드백 분석, 데이터 정리 및 분석, 법률 및 금융 문서 자동화 등의 기능 제공.

Copy.ai

🚀 Copy.ai의 주요 기능

1. 자동 콘텐츠 생성

- 블로그 글, 기사, 광고 카피, 제품 설명, 이메일 등 다양한 콘텐츠 자동 생성

2. 마케팅 카피 작성

- Facebook, Google Ads, LinkedIn 등의 광고 문구 생성

3. SEO 최적화

- 검색 엔진 친화적인 콘텐츠 추천 및 자동 완성

4. SNS 콘텐츠 작성

- 트위터, 인스타그램, 링크드인 등의 소셜 미디어 포스트 자동 생성

5. 번역 및 스타일 변환

- 문장을 원하는 스타일(전문적, 캐주얼, 창의적)로 변환 가능

🚀 Copy.ai와 LLM의 관계

Copy.ai는 GPT 계열의 대규모 언어 모델(LLM)을 기반으로 작동하며, OpenAI의 GPT-4 또는 자체 최적화된 모델을 사용하여 콘텐츠를 생성합니다.

💡 누가 사용하면 좋을까?

✅ 마케터, 콘텐츠 크리에이터, 스타트업, 이커머스 사업자 등이 빠르게 콘텐츠를 생성할 때 유용

비슷한 서비스로 **Jasper.ai**, **Writesonic**, **Rytr** 등이 있습니다. 😊

국내 심리상담

위로미 V2

닥터송이 개발한 정신건강 키오스크로, 사용자가 음성으로 고민을 이야기하면 이를 분석하여 상담 답변을 제공합니다. 다양한 상담 버전(명언, 인사임당, 미래의 나 등)을 통해 사용자에게 맞춤형 상담 경험을 제공합니다.

INEURO.KOREA.AC.KR



민음

KT가 개발한 초거대 AI 기반 심리케어 서비스로, 대국민 맞춤형 심리케어와 전문가용 심리치료 방향 제시 등을 제공합니다. 이를 통해 우울증, 자살 등 사회 문제 해결과 상담 인력 부족 문제를 완화하는 것을 목표로 합니다.

ETNEWS.COM



익시젠

LG유플러스의 초거대 AI 기술을 활용한 심리케어 서비스로, 대민 및 전문가 보조 AI 서비스를 개발·실증하여 정신건강 관리와 상담 효율성을 높이는 것을 목표로 합니다.

ETNEWS.COM



헬로우봇

딩스플로우에서 개발한 감성 기반 대화형 테라피 챗봇으로, 사용자와의 대화를 통해 심리 상담과 정서적 지원을 제공합니다.

STATIC4.APUB.KR



상냥이

AI 기반 청소년 상담 플랫폼으로, 자연어 처리 기술을 활용하여 청소년들의 고민과 문제를 상담하고 지원합니다.



Inference(추론) 계층

추론 단계의 활용 예시:

1. **챗봇:** 고객 서비스 챗봇이 사용자의 질문에 답하는 과정에서 추론을 사용합니다. 사용자가 질문을 입력하면, AI 모델은 해당 질문에 적합한 답변을 실시간으로 제공하게 됩니다. 예를 들어, OpenAI의 GPT 모델이나 Hugging Face의 모델이 추론 기능을 통해 챗봇 응답을 생성합니다.
2. **자동 완성 및 코딩 보조:** GitHub Copilot 같은 AI 코딩 도구는 개발자가 코드를 입력할 때 실시간으로 코드 제안을 제공합니다. 이 역시 추론 과정을 통해 이루어지며, 사용자의 입력을 분석한 후 가장 적합한 코드를 제안하거나 자동으로 완성합니다.
3. **자동 문서 요약:** 대규모 언어 모델은 긴 문서를 입력받아 요약본을 실시간으로 제공할 수 있습니다. 이는 비즈니스에서 문서 작업을 간소화하고 중요한 정보를 빠르게 얻는 데 매우 유용합니다.

추론(Inference)을 지원하는 주요 플랫폼:

이미지에서 언급된 **OpenAI**와 **Hugging Face**는 대표적인 추론 서비스 제공업체입니다.

- **OpenAI:** GPT 모델과 같은 대규모 언어 모델을 클라우드 기반으로 배포하여, 개발자와 기업이 실시간으로 모델을 활용할 수 있도록 추론 API를 제공합니다. 이를 통해 누구나 대규모 언어 모델을 활용해 다양한 응용 프로그램을 만들 수 있습니다.
- **Hugging Face:** **Transformers 라이브러리**와 **Inference API**를 통해 사용자가 자신의 데이터에 맞게 사전 학습된 모델을 사용하거나 커스터마이징하여 실시간으로 추론을 할 수 있게 도와줍니다. Hugging Face는 다양한 언어 모델을 지원하며, 이를 쉽게 활용할 수 있는 툴을 제공하는 것이 특징입니다.

추론 과정은 다음과 같이 진행됩니다:

1. **모델 학습 후 배포:** 모델은 데이터셋을 학습하여 문장 이해, 생성, 번역, 요약 등 여러 가지 언어 관련 작업을 할 수 있는 능력을 얻습니다. 이 학습이 완료되면 모델은 배포되어 사용될 준비가 됩니다.
2. **실시간 데이터 입력:** 추론 단계에서는 사용자가 입력한 텍스트나 질문 같은 실시간 데이터를 모델에 제공합니다. 예를 들어, 고객 서비스 챗봇이 사용자의 질문을 받거나, 코딩 도구에서 코드를 입력하는 상황을 생각할 수 있습니다.
3. **모델 추론:** 입력된 데이터를 바탕으로 모델은 학습된 내용을 활용하여 적절한 답변이나 출력을 생성합니다. 예를 들어, 질문에 대한 답을 제공하거나, 입력된 문장을 요약하거나, 특정 코드를 완성할 수 있습니다.
4. **결과 제공:** 모델이 생성한 출력은 실시간으로 사용자에게 전달됩니다. 이 과정은 매우 빠르게 이루어져야 하며, 특히 실시간 애플리케이션(예: 챗봇, 추천 시스템, 자동 응답 시스템)에서 중요합니다.

Inference(추론) 계층

◆ Infrastructure Layer (인프라 계층)

LLM을 지원하는 하드웨어 및 모델 학습 관련 기술들을 포함.

1. Model Creation (모델 생성)

- **AI21 Labs, OpenAI, Adept, EleutherAI, Anthropic, Google AI**
- AI 모델을 직접 개발하고 학습시키는 기업들.
- OpenAI와 Google AI는 대표적인 LLM 연구기관.

2. Hardware (하드웨어)

- **NVIDIA, Habana, SambaNova, Cerebras, Graphcore**
- AI 모델 학습 및 실행을 위한 GPU, AI 칩 개발 기업들.
- NVIDIA의 GPU가 대표적인 AI 학습용 하드웨어.

3. Fine Tuning (미세 조정)

- **Google AI, OpenAI, Hugging Face, AWS**
- 사전 학습된 AI 모델을 특정 용도로 맞춤 조정하는 역할.
- 예를 들어, GPT-4를 특정 도메인(예: 의료, 법률)에 맞게 튜닝할 때 사용됨.

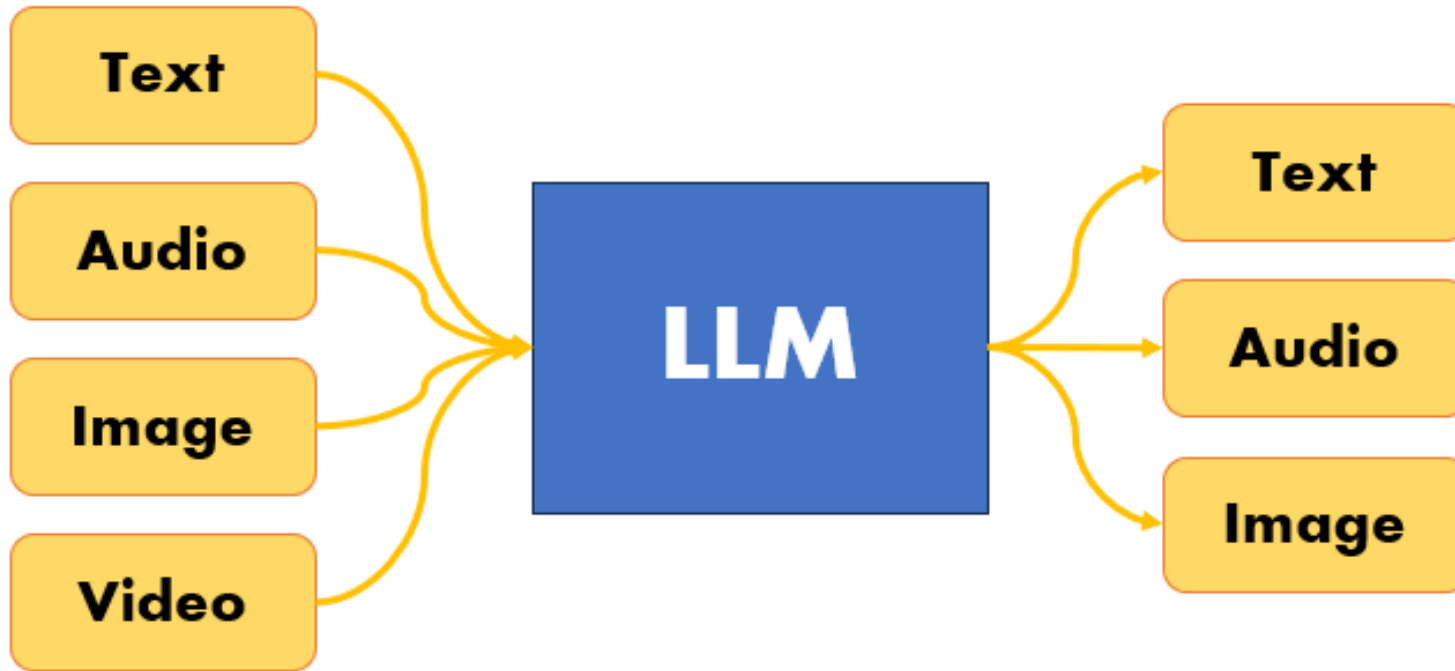
4. Inference (추론)

- **OpenAI, Hugging Face**
- 학습된 LLM을 기반으로 실시간 서비스 제공.
- OpenAI API를 활용한 다양한 AI 서비스가 대표적 사례.

7. 멀티모달

text, audio, image, video의 모든 조합을 입력으로 받아들이고, text, audio, image의 모든 조합을 생성한다.

멀티모달



GPT-4o는 기존 모델에 비해 vision 및 audio 이해 능력이 뛰어나다. GPT-4o는 text, audio, image, video의 모든 조합을 입력으로 받아들이고, text, audio, image의 모든 조합을 생성한다.

8. ChatGPT vs LLaMA vs Gemini vs Hugging Face

ChatGPT, LLaMA, Gemini, **허깅페이스(Hugging Face)**는 모두 인공지능(AI) 언어 모델 및 플랫폼 생태계의 중요한 요소들이며, 다음과 같이 분류될 수 있습니다:

1. ChatGPT, LLaMA, Gemini는 **대규모 언어 모델(LLM, Large Language Models)**입니다.

- 이들은 자연어 처리(NLP) 분야에서 사용되는 AI 모델로, 대규모 데이터로 학습되어 인간과 유사한 언어 이해와 생성 능력을 갖추고 있습니다.
- ChatGPT (OpenAI), LLaMA (Meta), Gemini (Google DeepMind)는 모두 특정 기업에서 개발한 언어 모델들입니다.

2. **허깅페이스(Hugging Face)**는 AI 플랫폼이자 커뮤니티 허브입니다.

- 허깅페이스는 대규모 언어 모델(LLM)을 포함한 다양한 AI 모델을 쉽게 사용할 수 있도록 도와주는 오픈 소스 플랫폼으로, AI 연구자와 개발자들이 모델을 공유하고 협력할 수 있는 공간을 제공합니다.
- Hugging Face는 자체적으로 AI 모델을 개발하기도 하지만, 주로 모델 허브를 통해 다양한 언어 모델(예: GPT, BERT, LLaMA, BLOOM)을 제공하며, 누구나 쉽게 활용할 수 있도록 Transformers 라이브러리와 같은 도구를 제공합니다.

- ChatGPT, LLaMA, Gemini: **대규모 언어 모델(LLM)**로, 각각 OpenAI, Meta, Google DeepMind에서 개발된 AI 모델들입니다.
- 허깅페이스(Hugging Face): 다양한 AI 모델을 위한 오픈 소스 플랫폼 및 커뮤니티로, 모델 개발, 배포, 공유를 지원하며, 대규모 언어 모델을 포함한 다양한 AI 모델을 쉽게 사용할 수 있도록 도와줍니다.

ChatGPT vs LLaMA vs Gemini vs Hugging Face

모델명	개발사/기관	특징 및 설명	주요 용도 및 특징
ChatGPT 	OpenAI	- GPT-3와 GPT-4 기반의 대형 언어 모델	- 대화형 AI 모델 - 다양한 언어 이해 및 생성 가능 - 텍스트 생성, 요약, 번역, 질의응답 등
LLaMA 	Meta (Facebook)	- "Large Language Model Meta AI"의 약자 - 연구 목적으로 개발된 오픈 소스 언어 모델 - 다양한 크기(7B, 13B, 30B, 65B 등)	- 학계 및 개발자들이 자유롭게 사용할 수 있는 오픈 소스 모델 - 텍스트 생성, 문서 요약, 질문 응답 등
Gemini 	Google DeepMind	- 이전 LaMDA와 PaLM 2 모델을 대체하는 차세대 AI 모델 - 2023년 12월 출시 - 다양한 크기와 버전 제공	- 구글의 대형 언어 모델 - 텍스트 생성, 코드 작성, 이미지 생성, 다중 모달 활용 등
Hugging Face  HUGGING FACE	Hugging Face	- 다양한 AI 모델을 제공하는 플랫폼 - 모델을 공유하고, 연구하는 오픈 소스 커뮤니티 - Transformer 기반 모델을 많이 제공	- 오픈 소스 모델 허브 - 다양한 NLP, 텍스트 생성, 번역, 감성 분석 등 모델 제공 - 학계 및 산업용

8. Chat GPT 와 API 요금제

항목	ChatGPT Plus	Team Plan	Enterprise Plan	GPT API
사용 위치	chat.openai.com	chat.openai.com	chat.openai.com	platform.openai.com (API 호출)
대상	개인	팀/조직	기업	개발자, 서비스 개발자
가격	\$20/월	사용자당 약 \$25~30/월	협상	토큰 단위 과금
GPT-4 사용	✓	✓	✓	✓ (단가 높음)
API 포함	✗	✗	일부 포함 가능	✓
커스터마이징	✗	기본 공유 공간	조직 정책, 보안	자유롭게 앱/웹/API에 통합 가능
대표 용도	개인 사용	팀 공동 사용	기업 내부 챗봇 등	외부 서비스/앱 개발용

✓ 추가 기능 (Team 전용)

- 팀 관리 대시보드
- 팀 GPT 공유
- 우선순위 처리 (API처럼 빠른 응답)
- 조직별 사용 통계

GPT 요금체계

특성	ChatGPT (무료 모델)	ChatGPT Plus (유료 모델)
모델 접근성	GPT-3.5 모델 사용 가능	GPT-4 모델 사용 가능 (최신 GPT-4o 모델 포함)
속도	상대적으로 느린 응답 속도	더 빠른 응답 속도 (우선 처리 대기열)
사용 제한	하루 사용량 및 요청 수에 제한	사용량 제한이 적음 (대기열에서 우선적으로 처리됨)
접속 우선 순위	트래픽이 많을 경우 접속 불가하거나 대기 시간이 길어짐	트래픽이 많을 때도 우선적으로 접속 가능
업데이트 및 기능	최신 기능과 모델에 대한 접근이 제한적일 수 있음	최신 기능과 업데이트를 우선적으로 이용 가능
대화 길이	대화 내역의 길이에 제한 있음	더 긴 대화 내역 처리 가능
가격	무료 사용	월 \$20 (구독료)
모델 성능	GPT-3.5 기반, 제한된 성능	GPT-4 기반으로 더 향상된 성능, 더 정확한 답변 제공
사용 대상	간단한 질문과 일상적인 대화	전문적인 질문, 고급 언어 작업 수행, 복잡한 대화 및 생성

GPT 요금체계

항목	GPT 무료 사용자	GPT Plus 사용자 (\$20/월)
사용 가능한 모델	GPT-3.5	GPT-4 (GPT-4-turbo 사용)
속도	보통	더 빠름
응답 품질	일반적인 수준	더 정확하고 고도화된 응답
사용 가능 시간	혼잡 시 사용 제한될 수 있음	우선 접속 보장
기능 (예: 코드 해석기, 이미지 생성 등)	일부 기능 제한됨	고급 기능 포함 (예: DALL·E, 코드 해석기, 파일 업로드 등)
GPT-4-turbo 사용 여부	✗	✓
업데이트 반영 속도	느림	최신 기능 우선 적용

GPT 요금체계

GPT Plus와 OpenAI API 사용의 차이점은 주로 목적과 과금 방식에 있습니다.

1. **GPT Plus:** 정액제 서비스로, 월 \$20에 ChatGPT에서 **GPT-4**를 사용할 수 있으며, **웹 인터페이스**에서 빠른 응답 속도와 우선 접근 권한을 제공합니다. 주로 개인 사용자에게 적합합니다.
2. **OpenAI API:** 사용량 기반 과금으로, 개발자가 **애플리케이션**에 GPT 모델을 통합해 원하는 서비스에 맞게 사용할 수 있습니다. 입력 및 출력 토큰 사용량에 따라 요금이 부과되며, 더 높은 유연성과 확장성을 제공합니다.

GPT API 요금체계

GPT 모델별 API 가격

모델명	입력 토큰 비용 (1,000 토큰당)	출력 토큰 비용 (1,000 토큰당)
GPT-3.5 Turbo	\$0.0005	\$0.0015
GPT-4	\$0.03	\$0.06
GPT-4 Turbo	\$0.015	\$0.03
GPT-4o	\$2.50	\$10.00
GPT-4o Mini	\$0.15	\$0.60
GPT-4.5	\$75.00	\$150.00

- **GPT-3.5 Turbo**: 비용 효율성이 높아 일반적인 텍스트 생성 작업에 적합.
- **GPT-4**: 더 높은 정확도와 복잡한 작업 처리가 가능하지만, 비용이 상대적으로 높다.
- **GPT-4 Turbo**: GPT-4의 성능을 유지하면서도 비용과 속도 면에서 개선된 모델.
- **GPT-4o**: 멀티모달 기능을 지원하며, 텍스트, 이미지, 오디오 처리가 가능하다.
- **GPT-4o Mini**: GPT-4o의 축소판으로, 비용이 저렴하면서도 다양한 기능을 제공.
- **GPT-4.5**: 가장 최신의 고성능 모델로, 창의적인 작업과 고급 분석에 적합하지만, 비용이 가장 높다.

API 모델별 비교

열 이름	의미
MODEL	사용 가능한 모델의 이름입니다.
TOKEN LIMITS	분당 토큰 처리 한도 (TPM: Tokens Per Minute)
REQUEST AND OTHER LIMITS	분당 요청 한도 (RPM: Requests Per Minute), 또는 기타 제한 (예: RPD: Requests Per Day)
BATCH QUEUE LIMITS	하루에 배치로 처리 가능한 최대 토큰 수 (TPD: Tokens Per Day)

MODEL	TOKEN LIMITS	REQUEST AND OTHER LIMITS	BATCH QUEUE LIMITS
gpt-4o	30,000 TPM	500 RPM	90,000 TPD
gpt-4o-mini	200,000 TPM	500 RPM 10,000 RPD	2,000,000 TPD
gpt-3.5-turbo	200,000 TPM	500 RPM 10,000 RPD	2,000,000 TPD
text-embedding-3-small	1,000,000 TPM	3,000 RPM	3,000,000 TPD
dall-e-3		500 RPM 5 images per minute	
tts-1		500 RPM	
whisper-1		500 RPM	

토큰(Token)

토큰은 텍스트를 구성하는 단위입니다. 하나의 토큰은 단어나 단어의 일부를 나타낼 수 있으며, 영어의 경우 일반적으로 1,000 토큰은 약 750단어에 해당합니다. 예를 들어, 문장 "ChatGPT is great!"는 4개의 토큰으로 계산됩니다:

1. "Chat"
2. "GPT"
3. "is"
4. "great!"

API 사용 시 입력 텍스트와 출력 텍스트의 토큰 수를 계산하여 요금이 부과됩니다.

OpenAI의 API 요금 체계는 모델에 따라 달라집니다. GPT-4 Turbo는 1M 입력 토큰당 \$0.03, GPT-4 32k는 1M 입력 토큰당 \$0.06입니다. GPT-3.5 Turbo는 1M 입력 토큰당 \$0.0015로 매우 저렴합니다. DALL-E 3 이미지 생성은 1024x1024 해상도 기준 \$0.04, Whisper 음성 모델은 1분당 \$0.006입니다.

토큰(Token)

토큰은 단순히 단어를 기준으로만 계산되지 않기 때문에 "1,000 토큰이 750 단어"라는 비율이 나옵니다. 토큰은 단어뿐만 아니라 단어의 일부(예: 접두사, 접미사)나 구두점, 공백도 포함할 수 있습니다. 특히 영어와 같은 언어에서 짧은 단어는 한 토큰으로 계산되지만, 긴 단어는 여러 토큰으로 나뉘질 수 있습니다. 예를 들어, "fantastic"과 같은 단어는 두 개의 토큰으로 처리될 수 있습니다.

따라서 문장의 복잡성이나 길이에 따라 1,000 토큰이 750 단어 정도로 변환되는 것입니다.

gpt-4o-mini

1. 토큰 제한 (200,000 TPM)

- 200,000 TPM은 분당 200,000개의 토큰을 사용할 수 있다는 뜻입니다.
- 신문 기사에서 한 줄은 대개 20~30개의 단어로 이루어집니다.
- 영어를 기준으로 1단어 = 1토큰 정도로 계산할 수 있습니다.
 - 예를 들어, 한 문장이 20개의 단어라면, 그 문장은 약 20개의 토큰이 됩니다.
 - 그러므로 200,000 TPM은 매 분마다 10,000개의 문장을 처리할 수 있는 양입니다 ($200,000 \div 20 = 10,000$).

신문 기사 예시:

- 한 신문 기사가 300~500단어 정도 된다고 가정해 보겠습니다.
 - 한 기사는 약 300~500토큰에 해당합니다.
 - 200,000 TPM으로는 분당 약 400개의 신문 기사를 처리할 수 있습니다 ($200,000 \div 500 = 400$).

gpt-4o-mini

2. 요청 제한 (500 RPM, 10,000 RPD)

- **500 RPM:** 매 분마다 500번의 요청을 보낼 수 있다는 의미입니다.
 - 예를 들어, 하나의 요청으로 1개의 신문 기사를 처리한다고 하면, 매 분마다 최대 500개의 기사를 처리할 수 있습니다.
 - 즉, 한 분에 500개의 신문 기사를 처리할 수 있는 속도입니다.
- **10,000 RPD:** 하루에 10,000번의 요청을 보낼 수 있습니다.
 - 하루 동안 10,000번의 요청이 가능하므로, 하루에 최대 10,000개의 신문 기사를 처리할 수 있습니다.

3. 배치 제한 (2,000,000 TPD)

- **2,000,000 TPD**는 하루에 2,000,000개의 토큰을 사용할 수 있다는 의미입니다.
 - 하루에 신문 기사 4,000개를 처리할 수 있습니다. ($2,000,000 \div 500 = 4,000$)
 - 즉, 하루 동안 약 4,000개의 신문 기사를 처리할 수 있습니다.

TTS-1 , Whisper-1

1. TTS-1 (Text-to-Speech)

- TTS-1은 텍스트를 음성으로 변환하는 모델입니다. 즉, 입력된 텍스트를 읽어주는 기능을 합니다.
- 500 RPM은 매 분마다 500번의 요청을 처리할 수 있다는 의미입니다.
 - 한 번의 요청으로 한 개의 텍스트를 음성으로 변환할 수 있다고 가정할 때, 매 분에 최대 500 개의 텍스트를 음성으로 변환할 수 있습니다.

신문 예시:

- 만약 한 신문 기사를 텍스트로 입력하고 음성으로 변환한다면, 500개의 신문 기사를 1분에 음성으로 변환할 수 있습니다.
- 하루에 500 RPM의 속도로 24시간 운영하면, 하루에 최대 720,000개의 기사를 음성으로 변환할 수 있습니다 ($500 \times 60 \times 24$).

2. Whisper-1 (Speech-to-Text)

- Whisper-1은 음성을 텍스트로 변환하는 모델입니다. 즉, 입력된 음성을 텍스트로 변환하는 기능을 합니다.
- 500 RPM은 매 분마다 500번의 요청을 처리할 수 있다는 의미입니다.
 - 한 번의 요청으로 1분 분량의 음성을 텍스트로 변환할 수 있다고 가정할 때, 매 분에 최대 500 개의 음성을 텍스트로 변환할 수 있습니다.

- **TTS-1: 1분에 최대 500개의 신문 기사를 음성으로 변환할 수 있습니다. 하루 동안 최대 720,000개의 신문 기사를 음성으로 변환할 수 있습니다.**
- **Whisper-1: 1분에 최대 500개의 음성 파일을 텍스트로 변환할 수 있습니다. 하루 동안 최대 720,000개의 음성 파일을 텍스트로 변환할 수 있습니다.**

OpenAi api Key 생성1

<https://platform.openai.com/api-keys>

계정아이콘클릭

API keys

As an owner of this organization, you can view and manage all API keys in this organization.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	PROJECT ACCESS	CREATED BY	PERMISSIONS
mykey	sk-...p5kA	Default project	it k	All

it k
ynaihk@gmail.com

⚙️ 🌙 💬

Your profile
Terms & policies
Log out

OpenAi api Key 생성2

SETTINGS

Your profile

ORGANIZATION

General

API keys

Admin keys

Members

Projects

API keys

+ Create new secret key

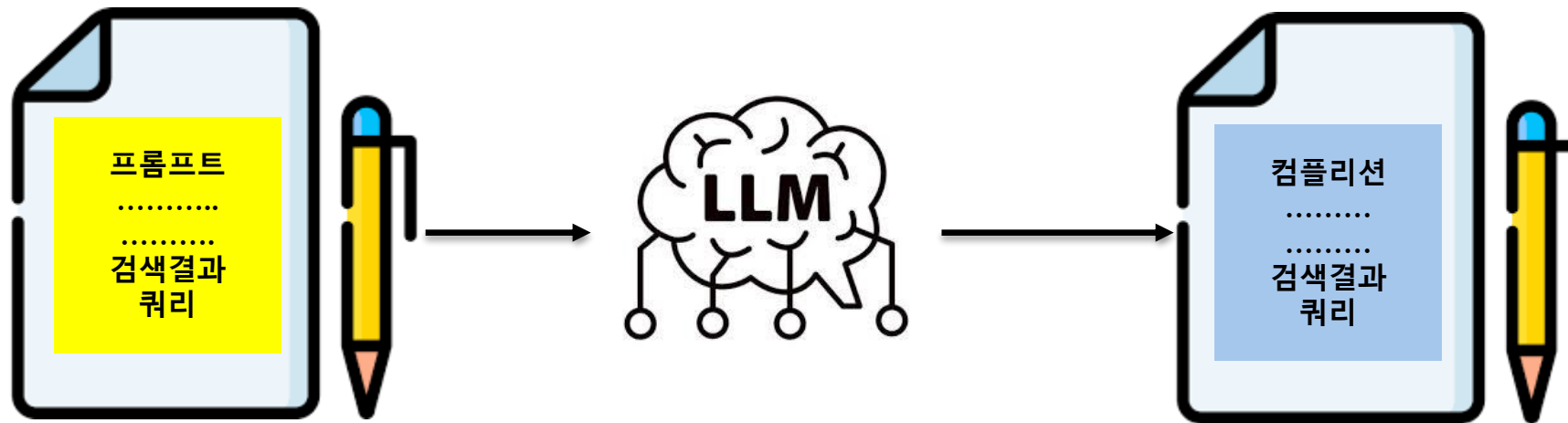
As an owner of this organization, you can view and manage all API keys in this organization.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	PROJECT ACCESS	CREATED BY	PERMISSIONS	
mykey	sk-...p5kA	Default project	it k	All	 

9. Prompt Engineering



질문: 프롬프트(prompt)

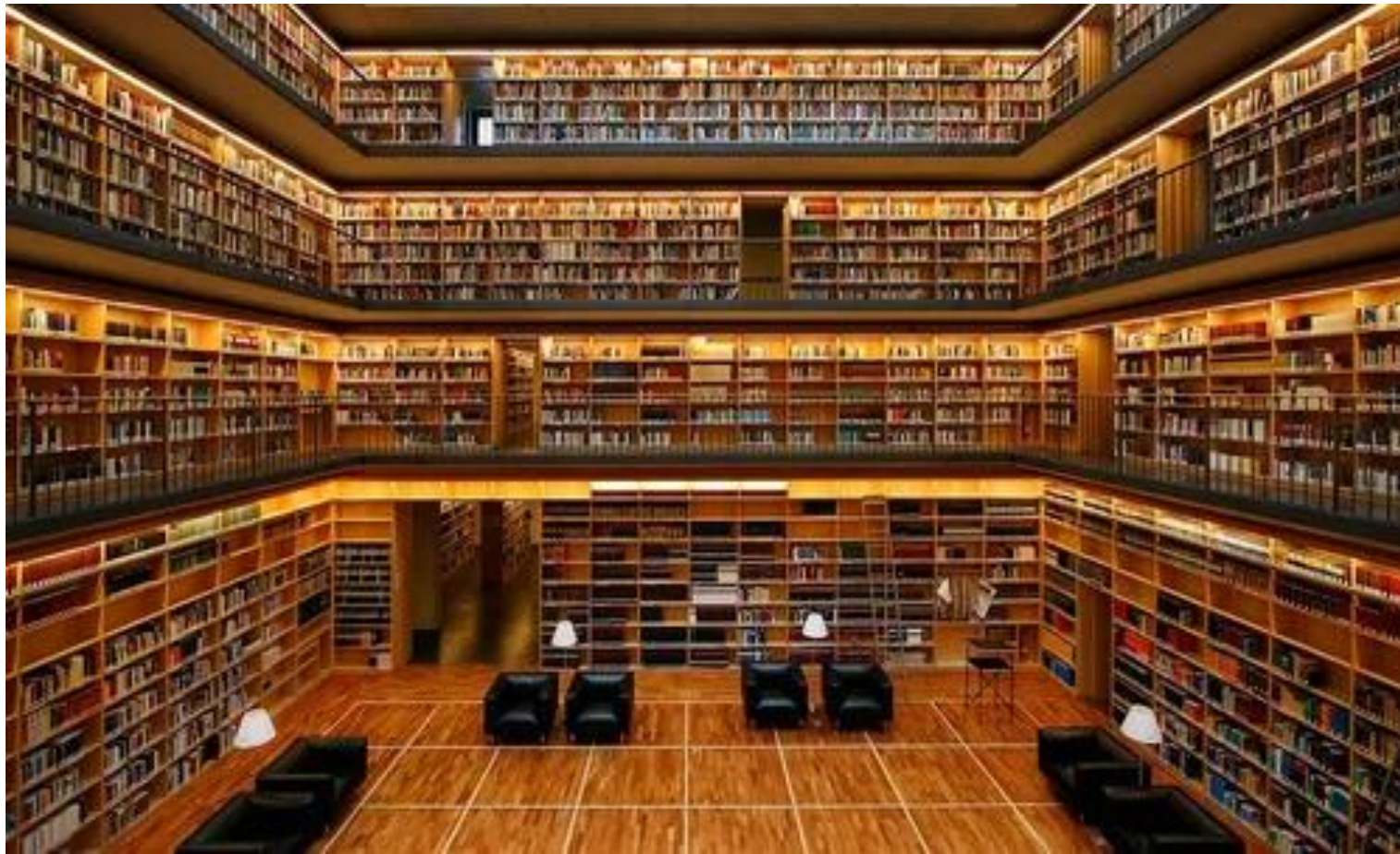
답변: 컴플리션(completion)

기본 원칙

평균 적인 답변을 얻을 것인지 구체적인 답변을 얻을 것인지 전문적인 답변을 얻을 것인지?

내가 원하는 그 구체적인 맥락에 chat gpt가 attention(집중) 하도록 유도를 해서 해당 맥락내에서 전문적이고 창의적 답변을 얻도록 해야함

GPT는 거대 도서관



GPT 는 모든 것을 알고 있지만 일반적인 질문을 하면 일반적 답변을 한다.

특정 전문 서적을 꺼내서 답변을 얻도록 질문을 유도해야 한다.

프롬프트 엔지니어링의 기본 원칙

명확성

날씨가 어때? (X)

서울의 현재 날씨 가 어때? (O)

상세성

피자 만드는 방법 은? (X)

불고기 피자를 집 에서 만드는
방법 은? (O)

문맥 이해

그 사람 누구야? (X)

앞에서 말한 20대 남자가 누구
야? (O)

프롬프트 방법

GPT 에게 문제를 명확하게 전달

문제 해결을 명확하게 이해 시키기(제약조건, 문자의 맥락, 작업내용)

GPT 전문성을 유도

전문성을 GPT 에게 지정(전문 직책명, 전문용어, 논문명)

GPT 성공적인결과 유도

형식과 구성을 알려주기 (예시를 주기)

전문가 역할부여

주식분석을 하는 업무에 적합한 전문가는 누구일까? 어떤 종류의 전문가가 있는지 알려줄래

Chat GPT 는 다양한 전문성을 가지고 있으나 그 전문성을 모르는 전문가이다. 전문성에 집중하도록 전문성을 끌어내는 prompt 필요, 전문성을 끌어내도록 가이드

(awesome GPT: 전문가 목록:<https://www.awesomegptprompts.com/>)

11개 프롬프트 엔지니어링 기법

- ▶ OpenAI, Google, Stanford, Princeton, DeepMind 등 대형 연구기관이나 AI 모델 제작사들이 논문을 통해 제안한 것들
- ▶ OpenAI는 GPT 시리즈(GPT-2, 3, 3.5, 4)를 만들면서 Zero-shot / Few-shot prompting 등 의 효율성을 실험한 결과
- ▶ 이후 외부 기관(Google, Princeton, DeepMind 등)도 참여해서
- ▶ 다양한 프롬프트 엔지니어링 전략 (CoT, ReAct, ToT 등)을 제안하였음

프롬프트 엔지니어링 기법 정리1

분류	기법 이름	설명
 기본	Zero-shot	예시 없이 바로 작업 지시
 기본	One-shot	예시 1개 제공 후 유사 작업 요청
 기본	Few-shot	예시 여러 개 제공
 맥락 제어	역할 부여(Role prompting)	모델에게 역할을 부여해 응답 방식 유도
 단계 유도	Chain-of-Thought(CoT)	모델이 단계별로 사고 과정을 먼저 쓰고 답을 내리도록 유도
 문제 분해	Tree-of-Thought(ToT)	다양한 해결 경로를 트리처럼 확장하며 사고, 비교 후 최적해 선택
 반복 개선	ReAct	모델이 **생각(Reasoning)** 과 **행동(Action)** 을 번갈아 반복하며 문제 해결

프롬프트 엔지니어링 기법정리 2

 제어 지시	Instruction Tuning	명령형 프롬프트를 기반으로 모델 훈련 or 사용 시 그에 맞는 형식으로 지시
 자기 피드백	Self-Refinement / Self-Critique	모델에게 자신의 응답을 검토하거나 스스로 개선하게 유도
 출력 제어	Output Priming / Formatting	출력의 형식, 스타일, 구조 등을 사전에 지정해서 일관성 유지
 질의 최적화	Prompt Chaining / Modular Prompting	여러 개의 프롬프트를 연쇄적으로 사용하여 복잡한 작업 처리

1. 제로 샷 프롬프트 (Zero-Shot)

- 모델에게 별도의 예시(example) 없이 바로 작업을 요청하는 프롬프트 엔지니어링 기법
- 문제에 대한 맥락이나 정답 예시 없이도 모델이 바로 이해하고 수행할 수 있도록 요청하는 방식
- 정의: 사전 예시 없이, 질문 또는 요청만으로 모델이 스스로 답변을 생성하도록 유도하는 방식
- 활용 예: 텍스트 요약, 감정 분석, 문장 평가, 아이디어 제안 등

1-1. 제로 샷 프롬프트 (Zero-Shot) 예

✅ 예시

💬 사용자:

arduino

📋 복사

✎ 편집

다음 문장을 영어로 번역해줘: "나는 오늘 커피를 두 잔 마셨어."

🧠 제로 샷 프롬프트의 형태:

pgsql

📋 복사

✎ 편집

Translate the **following** sentence **into** English: "나는 오늘 커피를 두 잔 마셨어."

👉 이처럼 번역하라고만 했지, 번역 예시는 주지 않았기 때문에 "제로 샷"이라고 부릅니다.

2. 원 샷 프롬프트 (One-Shot)

예시를 **1개만 제공**한 뒤, 유사한 작업을 요청하는 방식

2-1.원 샷 프롬프트 (One-Shot) 예

✅ 예시: 감정 분석 요청

💬 프롬프트:

makefile

📄 복사

✎ 편집

다음 문장의 감정을 분류해줘.

예시 :

문장: "오늘은 너무 기분이 좋아!"

감정: 긍정

문장: "하루 종일 비가 와서 무울해."

감정 :

🔍 설명:

- 첫 문장은 모델이 학습할 수 있도록 예시로 제공
- 두 번째 문장에 대해서는 모델이 패턴을 보고 감정을 추론함

3. 퓨샷 프롬프트 (Few-Shot)

- 예시를 **여러 개 제공**하고, 유사한 작업을 요청하는 방식
(일반적으로 2~5개 정도가 적당).

3-1. 퓨샷 프롬프트 (Few-Shot) 예

✅ 예시: 언어 감지

💬 프롬프트:

makefile

📄 복사

✎ 편집

다음 문장이 어떤 언어인지 감지해줘.

예시:

문장: "Bonjour tout le monde"

언어: 프랑스어

문장: "Guten Tag, wie geht's?"

언어: 독일어

문장: "こんにちは、元気ですか？"

언어:

apple: fruit

orange: fruit

zucchini: vegetable

tomato:

🔍 설명:

- 모델은 앞의 예시를 보고 패턴을 학습
- 유사한 방식으로 마지막 문장의 언어를 추론함

정리표

프롬프트 방식	예시 수	설명
Zero-shot	0개	예시 없이 바로 요청
One-shot	1개	예시 1개 + 새로운 입력
Few-shot	여러 개	예시 여러 개 + 새로운 입력

4. 역할 부여 프롬프트

모델에게 "너는 이제 ~~한 역할이야" 라고 지정해주는 프롬프트 방식

→ 모델이 그 역할에 맞춰 말투, 정보 수준, 응답 방식 등을 바꾸게 됨

☑ 기본 구조

너는 이제 [역할]이야. 너의 임무는 [목표/작업]이야.

4-1. 역할 부여 프롬프트 예

예시 1: 번역가 역할

arduino

너는 전문 영어 번역가야. 아래 문장을 자연스럽게 영어로 번역해줘. 문장: "요즘 날씨가 참 좋네."

예시 2: 초등학생 선생님 역할

너는 초등학교 4학년 학생에게 설명하는 과학 선생님이야. 블랙홀에 대해 쉽게 설명해줘.

예시 3: 까다로운 면접관 역할

너는 냉정하고 논리적인 면접관이야. 내가 답한 내용을 바탕으로 꼬리 질문을 던져줘.

🧩 왜 역할 부여가 유용할까?

장점	설명
🗨️ 맥락 제시	모델이 어떤 톤과 깊이로 응답할지 정할 수 있음
🎯 목적 명확화	작업의 목적과 기대 결과를 명확하게 전달 가능
🧠 정밀 제어	창의성, 전문성, 난이도 등을 조절하기 좋음

📁 팁

- 역할을 구체적으로 쓸수록 좋음
 - ❌ "전문가야" → 모호함
 - ✅ "10년 경력의 데이터 사이언티스트야"
- 필요하면 목표 + 조건도 같이 줘
 - 예: "간결하게 대답하고, 용어를 설명해줘"

5. Chain of Thought (CoT)

모델이 복잡한 문제를 단계적으로 생각하게 유도하는 기법.

"정답을 바로 묻지 말고, 먼저 생각하게 하자!"

- 모델에게 사고 과정을 먼저 서술하도록 유도하고
- 그 흐름을 따라 **최종 답을 도출**하게 함
- 특히 논리, 수학, 추론 문제에서 효과적임

CoT 기본 예시 구조

Q: [질문] A: 생각해보자. [1단계 사고] → [2단계 사고] → ... → 정답은 [답].

5-1. Chain of Thought (CoT) 과정

CoT 유도 질문의 핵심은?

"정답만 말하지 말고, 생각 과정을 단계별로 설명해줘" 라고 요청하는 것!

템플릿처럼 사용할 수 있는 CoT 유도 문장

diff

 복사

 편집

- 생각하는 과정을 단계별로 설명해줘.
- *Step-by-step reasoning*을 보여줘.
- 먼저 사고 과정을 서술한 다음에 정답을 말해줘.
- 중간 계산 과정까지 포함해서 설명해줘.

5-2. Chain of Thought (CoT) 예시

일반 질문 (Zero-shot)

사과가 3개 들어있는 바구니가 4개 있다. 총 사과는 몇 개인가?

☞ 이렇게만 하면 GPT는 그냥 "12개입니다." 라고 답할 가능성이 크다

Chain of Thought 유도 질문

사과가 3개 들어있는 바구니가 4개 있다. 총 사과는 몇 개인지, 생각하는 과정을 단계적으로 설명해줘.

5-2. Chain of Thought (CoT) 코딩활용예

Q:
주어진 정수 배열에서 짝수만 골라 새로운 리스트로 반환하는 함수를 만들어줘.
단계적으로 어떻게 접근할 수 있을지 설명한 뒤, 코드를 작성해줘.

Q: 두 리스트의 교집합을 구하는 가장 효율적인 방법을 생각해보고,
단계별로 전략을 설명한 후 파이썬 코드로 구현해줘.

6. Tree-of-Thought (ToT)

여러 가지 해결 방법(생각들)을 나무(Tree)처럼 분기하여 탐색하고, 그 중에서 가장 나은 해결책을 선택하거나 결합하는 프롬프트 기법

핵심 아이디어

- 한 번에 한 해답을 강요하지 않고, 다양한 아이디어를 펼쳐서 평가함
- CoT가 한 갈래의 생각 흐름이라면, ToT는 여러 갈래를 동시에 생각함
- 브레인스토밍, 전략 비교, 창의적인 아이디어 생성 등에 유리함

6-1. Tree-of-Thought (ToT) 기본구조

ToT 프롬프트 템플릿 예시

- 가능한 해결책 /아이디어를 3가지 제시해줘.
- 각각의 장단점 또는 조건을 비교해줘.
- 그 중 최선 또는 결합 가능한 아이디어를 선택해줘.

6-2. Tree-of-Thought (ToT) 예1

1. 창의적인 글쓰기 아이디어 생성

Q: "미래의 학교"를 주제로 단편소설을 쓰려고 해.

어떤 방향으로 쓸 수 있을지 아이디어를 3가지 제시하고, 각각 장단점을 설명해줘.

A:

1. 미래의 학교는 AI 선생님이 가르친다 → 기술 중심, 흥미로움 ★★★
2. 학교가 가상현실 공간이다 → 몰입도 높지만 현실성 낮음 ★★☆
3. 모든 학생이 개별 맞춤형 커리큘럼을 받는다 → 교육 혁신 주제에 적합 ★★★

결론: 1번과 3번을 조합해서 이야기하면 현실성 + 창의성을 모두 잡을 수 있을 것 같아.

6-3. Tree-of-Thought (ToT) 예2

2. 프로그래밍 문제 해결 전략 비교

Q: 정렬되지 않은 정수 리스트에서 중복을 제거하려고 해.

가능한 방법 3가지와 각각의 시간복잡도를 알려줘. 최선의 방법도 선택해줘.

A:

1. 리스트를 set으로 변환 → $O(n)$, 간단하고 빠름
2. 반복문으로 직접 검사 → $O(n^2)$, 비효율적
3. 정렬 후 인접 값 비교 → $O(n \log n)$, 중복 제거 가능

최선: set을 사용하는 1번 방법이 가장 효율적이다.

6-4. CoT vs ToT

💬 CoT와 ToT 비교 한눈에 보기

항목	Chain of Thought (CoT)	Tree of Thought (ToT)
사고 방식	한 가지 흐름으로 단계적 사고	여러 아이디어를 분기하여 평가
목적	논리적 추론, 계산	창의적 문제 해결, 선택과 비교
예시	수학, 로직 문제	전략 비교, 브레인스토밍, 선택

7. ReAct

"Reasoning(추론)"과 "Acting(행동)"을 번갈아 수행하는 프롬프트 기법

GPT가:

- 먼저 **생각(Reason)** 을 하고,
- 그다음 **행동(Action)** 을 하며,
- **환경에서 얻은 관찰 결과(Observation)** 를 바탕으로
- 다시 **생각 + 행동**을 반복함

이렇게 루프를 돌면서 더 정확하고 상황에 적응하는 방식으로 문제를 해결

7-1. ReAct 기본구조

ReAct 기본 구조

질문: [문제]

Thought: [이 문제를 어떻게 풀지 생각함]

Action: [할 행동을 정함]

Observation: [그 행동을 했을 때 얻은 정보]

... (필요시 반복)

Answer: [최종 답]

7-2. ReAct 예1

"현재 서울의 날씨를 알고 싶어. 아래 형식에 따라 단계적으로 생각하고 행동해줘:

- **Thought:** 먼저 어떤 정보가 필요한지 생각해보고
- **Action:** 어떤 행동을 할지
- **Observation:** 행동 결과를 바탕으로 얻은 정보
- **Answer:** 최종 답변

위 구조를 따라줘."

질문: 파일에서 중복된 이메일 주소를 제거하는 가장 효율적인 방법은?

ReAct 형식으로 생각하고 답해줘.

(Thought → Action → Observation → Answer 순으로)

7-2. ReAct 예2

이 코드가 너무 복잡해. 리팩토링할 수 있을까?
ReAct 형식으로 단계별로 개선 방향을 알려줘.

- Thought: 어떤 문제가 있는지 생각
- Action: 개선 가능한 부분 확인
- Observation: 개선 후 코드 구조의 변화
- Answer: 리팩토링된 코드와 설명

아래 프로그램에서 어떤 예외 처리를 추가해야 할까?
ReAct 형식으로 설명해줘.

- Thought: 어떤 에러가 발생할 수 있을지 예측
- Action: 코드 검토 or 실행 시나리오 상상
- Observation: 발생 가능한 예외 분석
- Answer: 예외 처리 추가 방식과 예시 코드

7-3. ReAct 적용시기

✨ 언제 ReAct를 쓰면 좋을까?

상황	효과
복잡한 추론이 필요할 때	단계적으로 생각하며 문제 해결
외부 환경이나 도구 활용이 필요할 때	검색, 계산기, API 등과 연계
디버깅, 시뮬레이션	관찰→행동 루프에 적합
에이전트(Agent) 설계	ReAct는 LangChain, AutoGPT 등의 핵심 로직 기법

8. 제어 지시 (Instruction Prompting)

모델에게 "무엇을 어떻게 하라"는 명령어(instruction)를 명확히 주는 프롬프트 기법

- GPT 모델은 **지시에 잘 따르는** 특성이 있음
- 명확하고 구체적으로 지시하면 **일관된 결과**를 얻기 쉬움
- 사전 학습된 instruction-following 데이터에 기반한 응답 성향

8-1. 제어 지시 (Instruction Prompting) 형식

✨ 자주 쓰는 지시어 템플릿

- ~~을 ~~ 형식으로 설명해줘
- ~~한 말투로 요약해줘 (예: 공식적인, 친근한)
- ~~ 방식으로 정리해줘 (예: 단계별, 목록)
- ~~만 출력해줘 (예: JSON만, 코드만, 표만)
- ~~를 포함하지 말고 대답해줘 (예: 설명 없이 코드만)

8-2. 제어 지시 (Instruction Prompting) 예

"아래 문장을 영어로 번역하고, 결과는 큰따옴표로 감싸서 출력해줘."

입력: 나는 오늘 커피를 마셨다.

"다음 문장을 뉴스 기사 스타일로 요약해줘."

입력: 어젯밤 도심에서 소규모 화재가 발생했지만, 다행히 인명 피해는 없었습니다.

"초등학생도 이해할 수 있게 설명해줘.

'블록체인'이 뭐야?"

8-3. 제어 지시 (Instruction Prompting) 적용방법

언제 쓰면 좋을까?

상황	효과
출력 형식이 중요할 때	예: JSON, 마크다운, 표 등
대상/레벨에 맞는 설명이 필요할 때	예: 초보자, 전문가 등
다양한 프롬프트 스타일을 일관되게 유지하고 싶을 때	명령형 지시로 통일 가능
GPT에게 과업을 정확히 전달하고 싶을 때	"명확한 목적 + 출력 방식" 제시 가능

9. 자기 피드백(Self-Refinement)

모델이 자신이 생성한 응답을 스스로 평가하고,
오류나 부족한 점을 찾아 수정 또는 개선하는 프롬프트 기법

- 1차 응답을 만든 뒤 → 스스로 검토하고 평가 → 필요시 수정/보완
- 사람처럼 "내가 이렇게 답했는데, 괜찮은가?" 하고 자기 점검하는 방식
- GPT를 더 정교하고 신뢰할 수 있게 만드는 고급 기법

9-1. 자기 피드백(Self-Refinement) 형식

📌 형식 1: 답변 → 피드백 → 개선

1. 먼저 질문에 대한 답변을 해줘.
2. 그런 다음, 네가 작성한 답변을 스스로 평가해봐.
 - 정확성, 논리성, 표현의 적절성 측면에서
3. 필요하다면 답변을 수정하거나 보완해줘.

형식은 아래와 같이 따라줘:

 ****초기 답변**:**

[처음 작성한 응답]

 ****자기 피드백**:**

- [무엇이 좋았는지]
- [부족하거나 잘못된 부분]
- [개선 방향]

 ****개선된 답변**:**

[수정/보완된 응답]

예시 질문	사용 이유
"이 개념 설명이 정확한지 스스로 평가해줘."	개념 누락/오해 여부 점검
"이 코드에 문제가 없는지 네가 다시 확인해봐."	버그/비효율 제거
"너가 한 요약에 빠진 정보가 있는지 봐줘."	요약의 완성도 향상
"작성한 답변을 전문가 관점에서 평가해줘."	전문가 수준 피드백 시뮬레이션

9-2. 자기 피드백(Self-Refinement) 형식 예

Q: 블록체인이 무엇인지 설명해줘. 그리고 너의 설명이 정확한지 스스로 평가해봐.

→

답변: 블록체인은 분산된 디지털 장부 시스템이다...

피드백: 설명은 기본 개념을 잘 담고 있지만, '탈중앙화'나 '보안성'에 대한 언급이 부족하다. 이 내용을 추가해보자.

→ 개선된 답변: 블록체인은 탈중앙화된 분산 디지털 장부 시스템으로, ...

- ▶ 아이디어: 학교에 "조용한 명상실"을 만들자.
- ▶ 학생들이 스트레스를 받을 때 혼자 조용히 쉴 수 있는 공간이 필요해.
- ▶ 그래서 도서관 옆에 명상실을 만들어, 조용히 음악을 들으며 쉴 수 있게 하자.
- ▶ ---
- ▶ 이제 아래 질문에 따라 네 아이디어를 스스로 비판적으로 평가해보:
- ▶ 1. 이 아이디어는 실제로 실행 가능한가?
- ▶ 2. 대상 사용자(학생)에게 정말 도움이 될까?
- ▶ 3. 이보다 더 나은 방법은 없을까?

9-3. 자기 피드백(Self-Refinement) 적용방법

사용 상황	설명	기대 효과
 복잡한 설명을 생성한 후	모델이 작성한 설명이 정확하고 논리적인지 스스로 검토하게 함	정확성 향상, 오해 방지
 코드 작성 후	작성한 코드의 성능, 가독성, 오류 가능성을 자체 검토	코드 품질 개선, 디버깅 보조
 요약 생성 후	중요한 정보가 누락되었는지 확인하고 보완	더 완성도 높은 요약 생성
 창의적인 글쓰기나 에세이 작성 후	표현의 적절성, 구조의 논리성, 흐름 등을 점검	글의 품질 향상
 AI 에이전트 설계 시	AI가 스스로 결과를 점검하고 수정하도록 구성	더 신뢰도 높은 응답 생성
 피드백 없는 환경에서	사용자가 직접 검토하기 어려운 경우, AI가 자기 검토	자가 품질 관리 가능

10. 출력제어

GPT에게 **답변의 형식, 구조, 스타일 등을 명확히 지정해서** 원하는 방식으로 출력을 제어하는 프롬프트 기법

핵심 목적

- 일관된 출력 형식 확보
- 구조화된 데이터, 코드, JSON, 표 등 가공하기 쉬운 출력 생성
- 다양한 대상/플랫폼에 맞춘 스타일로 출력 유도

10-1. 출력제어 예

✓ 1. JSON 형식 출력 요청

text

아래 정보를 JSON 형식으로 출력해줘.
이름, 나이, 이메일을 포함해야 해.

출력 형식 예시:

```
{  
  "name": "",  
  "age": ,  
  "email": ""  
}
```

✓ 2. 표(Table) 형식 요청

text

파이썬의 리스트와 튜플을 비교해서 표로 정리해줘.
마크다운 형식의 표로 출력해줘.

✓ 3. 코드만 출력 (설명 없이)

text

문자열을 뒤집는 파이썬 코드를 보여줘.
설명 없이 코드만 출력해줘.
코드는 Python 코드 블록(```python)으로 감싸줘.

✓ 4. 마크다운 형식 요청

text

“AI의 장단점”을 제목으로 해서 마크다운 형식 문서를 만들어줘.
장점은 3가지, 단점은 2가지 항목으로 나눠서 작성해줘.

10-2. 출력제어 예

출력 제어 프롬프트 요약

목적	지시 문구 예시
JSON 출력	"JSON 형식으로 출력해줘"
표 출력	"표로 정리해줘", "마크다운 표 형식으로"
코드만	"설명 없이 코드만 출력해줘"
마크다운	"마크다운 문서 형식으로 보여줘"
길이 제한	"300자 이내로", "두 문장으로만"
태그 형식	"HTML/XML 형식으로 출력해줘"

11. 질의 최적화

GPT에게 더 정확하고 유용한 응답을 얻기 위해 질문(프롬프트)의 표현이나 구조를 개선하는 기법

💡 왜 필요할까?

- GPT는 질문의 방식에 따라 응답의 질이 크게 달라짐
- 따라서 의도를 명확하게 전달하고, 출력 형식이나 조건을 붙여서 더 나은 결과를 얻는 게 중요

11-1. 질의 최적화 형식

질의 최적화 프롬프트 템플릿

- "이 정보를 ~~형식으로 알려줘."
- "이 코드가 ~~하도록 하려면 어떤 문제가 있을까?"
- "~~의 관점에서 설명해줘."
- "단계별로 설명해줘."
- "아래 예시를 참고해서 ~~해줘."

11-2. 질의 최적화 예

1. ❌ 모호한 질문 → ✅ 명확한 요청

❌ “AI에 대해 알려줘.”

✅ “AI(인공지능)의 정의와 주요 기술 3가지를 간단히 설명해줘.”

→ 모호한 질문은 광범위한 답이 나올 수 있음 → 구체적으로 질문하면 응답 품질 향상!

2. ❌ 단순 요청 → ✅ 출력 형식 포함

❌ “파이썬으로 리스트 정렬 방법 알려줘.”

✅ “파이썬 리스트 정렬 방법을 코드 예제와 함께, 마크다운 코드 블록 형식으로 알려줘.”

→ 원하는 출력 형식을 알려주면 가독성 좋고 활용하기 쉬운 결과를 받을 수 있음

11-3. 질의 최적화 예

3. ❌ 한꺼번에 복잡한 질문 → ✅ 단계적 요청

❌ “AI 윤리 이슈와 해결 방안은 뭔데?”



1단계: “AI 윤리 문제의 대표적인 사례 3가지를 알려줘.”

2단계: “각 문제에 대한 현재 논의되고 있는 해결책도 알려줘.”

→ 질문을 나누면 더 깊고 정확한 응답을 받을 수 있음

4. ❌ 정보 부족 → ✅ 맥락 포함

❌ “이 코드에 버그 있어?”

✅ “이 코드는 사용자로부터 입력을 받아 리스트에 추가하는 기능을 해야 해. 아래 코드에서 버그가 있는지 확인해줘.”

→ 모델이 코드를 이해할 수 있도록 기대 동작이나 목적을 함께 제공

11-4. 질의 최적화 예

5. ❌ 목표 불분명 → ✅ 작업 목적 명시

❌ “이 문장을 자연스럽게 바꿔줘.”

✅ “이 문장을 비즈니스 이메일에 어울리는 공손한 표현으로 바꿔줘.”

→ 표현의 톤과 용도를 명확히 하면 더 원하는 스타일로 변환 가능

11-5. 질의 최적화 사용

정리: 질의 최적화는 언제 쓰면 좋을까?

사용 상황	기대 효과
모델이 잘못된 답을 줄 때	질문을 더 구체화하여 정확도 향상
출력이 너무 길거나 짧을 때	출력 형식이나 길이 제한을 추가
원하는 톤/스타일이 아닐 때	맥락 또는 대상 독자를 명시
복잡한 문제를 다룰 때	질문을 쪼개거나 순차적 사고 유도

프롬프트 구성 6가지요소

- ▶ AI Ground 에서 자체적으로 정리한 작성 가이드
- ▶ 프롬프트 구성 요소는 프롬프트 자체를 어떻게 구성할지에 대한 지침

프롬프트 구성 6가지요소

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

필수적 요소, 반드시 포함해야 함

맥락 [context]

페르소나 [persona]

권장, 있으면 높은 확률로 결과가 개선됨

예시 [example]

포맷 [format]

있으면 좋음

어조 [tone]

+ 모든 요소가 반드시 포함되어야 하는 것은 아님
(상황에 맞게 넣을 수 있는 요소들을 넣어주면 됩니다.)

*출처 : Jeff Su Youtube

명령 [task]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

- 명령어는 반드시 포함해야 합니다.
- 서술어로 명료하게 기술하세요. (ex. ~ 작성해줘, 요약해줘)

맥락 [context]

질 좋고 자세한 답변을 원한다면, 하나의 프롬프트에 한개의 Task만 쓰세요.

페르소나 [persona]

예시 [example]

- 가능하면 하나의 명령을 주도록
- 여러명령은 단계적으로 주도록 한다

포맷 [format]

어조 [tone]

맥락[context]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

맥락 [context]

페르소나 [persona]

예시 [example]

포맷 [format]

어조 [tone]

- 다음과 같은 관점에서 생각해 보는게 도움이 됩니다.
 - 어떤 상황/배경에 처해있는지?
 - 의도와 목표는 무엇인지?
 - 우려되는 점은 무엇인지?
 - 고려해야할 제약 사항/규칙은 어떤 것이 있는지?

페르소나[persona]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

맥락 [context]

페르소나 [persona]

예시 [example]

포맷 [format]

어조 [tone]



You

헬스장 홍보를 위한 전단지 내용을 작성해줘

X



You

헬스장 홍보를 위한 전단지 내용을 작성해줘

우리 헬스장의 이름은 '머슬베어'야. 우리 헬스장은 운동 초보를 위한 맞춤형 운동 프로그램을 제공하는 소규모 PT 전문 헬스장이야. 운동 초보자들이 쉽게 운동에 입문하고, 체계적으로 체중 감량, 자세 교정, 체력 증진 등 목표하는 운동 목표를 달성할 수 있게 맞춤형 지도를 제공하는 것을 가장 중요하게 생각하고 있어. 갑작스러운 체중 감량이나 바디 프로필 처럼 건강을 해칠수 있는 프로그램보다는 기초 대사량을 늘리고 장기적으로 고객들이 건강을 회복하고 활력있는 삶을 살도록 도와주는 것을 내 사명으로 생각하고 있어.

이번 전단지는 3월을 맞이해서 초등학교 개학을 하니까, 단지 내 초등학교, 중학생 학부모이신 30~40대 여성을 타겟으로 만들고 싶어. 개학으로 생긴 여유 시간에 운동을 하라고 독려하고 싶어. 그리고 미루던 새해 결심을 지금 실천할 때라는 것을 강조하고 싶어. 이 기간에 등록을 독려하기 위해서 18시 이전에 PT 예약하면 원래 가격인 시간당 5만원에서 4만원으로 할인 혜택을 제공한다는 내용도 포함하고 싶어.

O

페르소나[persona]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

맥락 [context]

페르소나 [persona]

예시 [example]

포맷 [format]

어조 [tone]

- 해당 문제를 가장 잘 해결할 수 있는 사람/역할/직무로 Role Play
- 구체적인 전문가 명칭일수록, 전문 용어를 포함할수록 더 전문적인 답변합니다.

페르소나[persona]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

맥락 [context]

페르소나 [persona]

예시 [example]

포맷 [format]

어조 [tone]



You

K푸드를 홍보하기 위한 유튜브 콘텐츠의 소재, 제목, 썸네일 문구를 작성해줘

X



You

너는 사람들에게 한국의 음식을 소개하고 추천해주는 유튜버야. 치킨, 떡볶이, 토스트처럼 한국 사람들이 즐겨먹지만 전세계에서는 잘 모르는 음식들로 콘텐츠를 만들기도 하고, 김치나 비빔밥처럼 한국 전통 음식들을 외국인들에게 먹여보고 리액션을 촬영하는 비디오 등으로 인기를 끌었어. 음식에 대한 재미있는 스토리를 만들어내거나, 한국 음식을 먹고 새로워하고 즐거워하는 외국인들의 리액션을 생동감 있게 제작해서 큰 인기를 얻었어. 너는 한국 관광공사의 광고비를 받고, 한국에서만 먹을 수 있는 맛있는 음식들을 미국에 있는 현지 미국인들에게 먹여보고, 한국에 대한 호의적인 반응들을 이끌어내서, 한국 음식에 대한 호기심을 불러일으키는 콘텐츠를 만들려고 해.

다양한 K푸드를 홍보하기 위한 유튜브 콘텐츠의 소재, 제목, 썸네일 문구를 작성해줘

O

예시 [example]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

맥락 [context]

페르소나 [persona]

예시 [example]

포맷 [format]

- 의외로 **항상 체감**이 가장 되는 것이 바로 '예시'입니다.
- 맥락에 잘 맞는 예시를 주는 것이 중요합니다.
- 예시가 길지 않다면 2개 이상을 주는 것도 좋습니다. (few shot)

The main idea of CoT is that by showing the LLM some few shot **exemplars where the reasoning process is explained in the exemplars**, the LLM will also show the reasoning process when answering the prompt. This explanation of reasoning often **leads to more accurate results**.

예시 [example]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

맥락 [context]

페르소나 [persona]

예시 [example]

포맷 [format]

어조 [tone]



You

당신은 민사소송 전문 변호사로 활동하고 있습니다. 반도체 구매대금지급을 미루는 바이어A에게 보낼 내용 증명 초안을 작성해 주세요.

X



You

당신은 민사소송 전문 변호사로 활동하고 있습니다. 반도체 구매대금지급을 미루는 바이어A에게 보낼 내용 증명 초안을 작성해 주세요. 내용 증명의 내용 구성, 형식, 어조는 예시의 내용 증명을 참고해서 마크다운 형식으로 작성해주세요.

[내용 증명 예시]

#내용 증명

##수신인

- 이름: xxx

- 주소: xx시 xx구 xxx

##발신인

- 이름: xxx

- 주소: xx시 xx구 xxx

##제목: 임대차계약 해지 통지

O

예시주는 방법: 텍스트, 파일, 인터넷링크



인터넷 링크 URL



파일
(PDF, Excel, Txt)



프롬프트 내 텍스트 입력

- 챗GPT 유료 모델 사용 시
 - URL 링크, 파일 첨부를 통한 예시/템플릿 제공 가능
- 챗GPT 무료 모델 사용 시
 - 참조 데이터를 직접 텍스트로 입력

포맷[format]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

맥락 [context]

페르소나 [persona]

예시 [example]

포맷 [format]

어조 [tone]



You

신입사원을 위한 1일 8시간 동안의 워크샵 프로그램을 작성해주세요. 주제는 '생성형 AI의 실무 활용'입니다. 아래의 구성대로 마크다운 형식으로 작성해주세요.

##1.워크샵 주제 :

##2.워크샵의 기대효과 :

##3.워크샵 목차:

##4.목차별 세부 강의 내용 및 시간배정 :

- 타임테이블 표 형식으로 작성하세요.
- 표의 열은 제목, 시간 배정, 과제로 구성해주세요.



You

내 사업 아이디어를 바탕으로 체계적인 사업 계획서를 작성해줘.

##내 사업 아이디어

인공지능을 활용해 각 채널에 맞는 콘텐츠를 자동으로 생성하는 IT 서비스를 만들고 싶습니다. 제가 블로그를 작성하면 채널의 톤앤매너에 맞게 원소스멀티유즈로 유튜브/인스타그램에 맞는 맞춤형 콘텐츠를 생성해 주는 것이죠. 각 채널에서 인기 있는 콘텐츠의 형식과 분량을 자동으로 변경해 주는 것이 특징입니다. 시청자를 확보하려는 크리에이터나 퍼스널 브랜딩에 관심이 있는 사람들을 타겟 고객으로 합니다.

##구조화된 사업 계획서 형식

- (1) 서비스 개요, (2) 타겟 고객, (3) 타겟 시장, (4) 주요 특징, (5) 차별화 전략

어조[tone]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

맥락 [context]

페르소나 [persona]

예시 [example]

포맷 [format]

어조 [tone]

- 형용사를 사용하세요. (ex. 간결하게, 친근하게...)
- 예시를 제공해서 어조를 모방하게 할 수 있습니다.

어조[tone]

구체적인 프롬프트를 구성하기 위한 6가지 구성요소

명령 [task]

맥락 [context]

페르소나 [persona]

예시 [example]

포맷 [format]

어조 [tone]

당신
고객 관점에서 이해하기 쉽고 친근하고 다정한 어조로 말하며, 간결하게 말하는 UX Writing의 원칙에 따라 아래의 글을 다시 써줘. 다시 쓸 글은 백틱 3개로 구분할게. 3개 정도의 후보를 같이 제안해줘.

UX Writing의 원칙에 대해서 이해할 수 있게 예시로 설명할게.

1. '오늘 하루 그만보기' → '내일 다시 보기'로 변경
- 가급적 긍정적인 화법으로 써줘.
2. '상품 상세' → '작품 설명'
- 미술 작품을 취급하는 앱의 경우, 사용자에게 미술작품이 그저 '상품'인 것이 아니라, '작품'으로 다가가는 서비스의 취지에 맞게 단어를 수정했어. 이 외에도 상품으로 작성한 단어들이 있는데, 이를 모두 '작품'으로 변경해 통일감을 주면 훨씬 더 앱 서비스의 취지에 어울릴 것 같아.
3. 'Coming Soon' → '곧 오픈합니다'
- 한국인 사용자를 대상으로 하기 때문에 영어보다는 쉬운 한국어를 써줘.
4. '내역 없음' → 현재 진행 중 내용이 없습니다
- 사용자에게 '상의'와 '진상'을 보여주기 위해 줄임말 대신 '현재 진행 중 내용이 없습니다'라는 정중한 문구를 사용했어.
5. 커뮤니티 활동 없음 → '활동 내용이 없습니다. 커뮤니티에 의견을 남겨보시겠어요?'
- 이해하기 쉬운 글로 설명하는 것이 사용자 이용성에 더 좋아. 여기에 사용자의 활동까지 유도할 수 있는 문장으로 개선했어.
6. '진행하시겠습니까?, 취소/확인' → '사용하시겠어요?, 아니요/네'
- 실제 대화를 하는 듯한 내용으로 개선했어. 진행 여부를 문의하고 확인이나 취소만이 보이는 경험보다는 문장으로 확실하게 사용에 대한 의견을 묻는 것이 더 좋은 경험을 제공할 것으로 생각해.

이 페이지에서 나가면 작성하신 내용이 저장되지 않습니다

예시로 설명

ChatGPT

UX Writing의 원칙을 적용하여, 사용자가 이해하기 쉽고 긍정적이며 친근한 어조로 느낄 수 있도록 아래 문구를 다시 작성해볼게요. 여기서 중요한 것은 간결함과 동시에 사용자의 행동을 유도하고 안심시킬 수 있는 어투를 사용하는 것입니다.

후보 1

"페이지를 벗어나면 입력하신 정보는 저장되지 않아요. 계속하시겠어요?"

후보 2

"잠깐, 여기서 나가시면 지금까지의 작성 내용이 사라져요. 그래도 괜찮으세요?"

후보 3

"주의해주세요! 이 페이지를 떠나면 작성한 내용이 저장되지 않습니다. 계속 진행할까요?"

이 세 가지 옵션은 모두 사용자에게 현재 상황을 명확하게 전달하면서, 그들의 결정을 존중하는 태도를 보여줍니다. 동시에 사용자가 다음 단계로 넘어갈지 여부를 고민하게 만들어, 실수로 중요한 정보를 잃어버리는 일이 없도록 돕습니다.

영어 질문 vs 한국어 질문

정보와 관련 있는 외국어로 지시하기

영어권 자료가 많은 분야
는 영어로

- "What are the ..."

(아주 전문화된) 국어 자료
가 필요한 분야는 국어

- "훈민정음의 의미는?"

영어 질문 vs 한국어 질문

영어와 한국어

- 영어 프롬프트가 더 길고 더 전문적
- 토큰이 적기 때문에 답변이 길고 더 전문적
- 영어 전문지식이 많음
- 한국적상황이 필요하면 한국어로

상호 협력

상호 협력 관점으로

- 어떤 전문가가 이 일을 잘 해결해 줄까
- 내 상황을 설명해주고 추가 적인 정보가 어떻게 있어

하나씩 단계적으로 대화 하면서 답변을 개선하면 높은 결과가 나온다

업무(domain)에 대한 충분한 지식

내 업무에 충분한 경험과 고민이 있으면 그 일에 대한 메타 인지 (경험과 고민)를 가지면 언어화 해서 프롬프트 작성이 좋은 결과가 나온다

내가 갖춰야할 3가지 기본 소양

나의 도메인지식과 전문성

인문학적 소양

센스, 감, 암묵지

10. 임베딩

임베딩(Embedding)

1. 임베딩이란 무엇인가?

- 복잡한 데이터를 일정한 차원의 벡터(숫자들의 집합)로 변환하는 과정.
- 우리가 사용하는 단어는 컴퓨터에게는 그저 문자들의 나열에 불과.
- 임베딩을 사용하면 각 단어를 숫자로 된 벡터로 바꿔 컴퓨터가 이해할 수 있게 해준다.

2. 왜 필요한가?

컴퓨터는 숫자만 이해할 수 있기 때문에, 텍스트나 이미지를 분석하려면 이들을 숫자로 변환해야 한다.

"사랑"이라는 단어를 컴퓨터에게 가르치려면, 그 단어를 고유한 숫자 벡터로 변환해야 한다. 이때, 임베딩은 "사랑"이라는 단어가 다른 단어들과 어떤 관계를 맺고 있는지, 즉 "사랑"이 "감정"이나 "우정"과 비슷하다는 것을 학습하게 해준다.

임베딩의 원리

- 임베딩은 고차원의 데이터를 저차원의 연속적인 벡터 공간으로 표현.
- 예를 들어, 수백 개의 단어를 각기 다른 벡터로 변환하면서도, 비슷한 의미의 단어들은 서로 가까운 위치에 있도록 배치
- 주로 딥러닝을 통해 이루어지며, 데이터 간의 관계를 학습하면서 벡터를 최적화.

예를 들어:

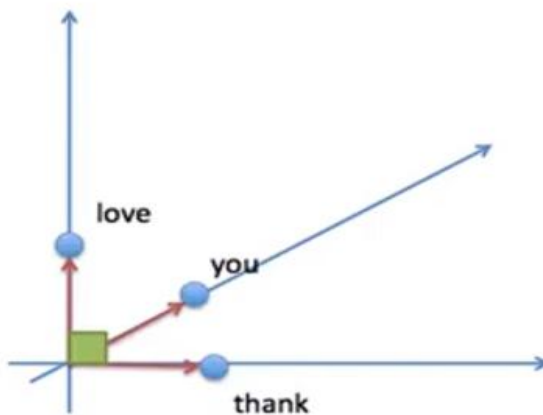
- "고양이"와 "강아지"는 둘 다 동물이라는 공통점이 있기 때문에, 임베딩된 벡터 공간에서는 두 단어의 벡터가 가까운 위치에 있을 것이다.
- 반면에 "고양이"와 "자동차"는 의미가 전혀 다르기 때문에 벡터 공간에서도 멀리 떨어져 있을 것이다.

원-핫 인코딩

- 1과 0으로 수치화
- 단어의 개수가 늘어나면 벡터의 차원이 한없이 커진다
- 이러한 벡터 표현은 공간적 낭비를 불러온다
- 원-핫 벡터는 단어의 의미를 담지 못한다
- 유사도분석 않됨

One Hot Encoding doesn't have **similarity**

cosine similarity also 0 since angle is 90 degree



unique word	encoding
thank	[1, 0, 0]
you	[0, 1, 0]
love	[0, 0, 1]

임베딩

원핫에 비해 저차원으로 표현 되며 유사도를 가진다

1-of-N Encoding

apple = [1 0 0 0 0]

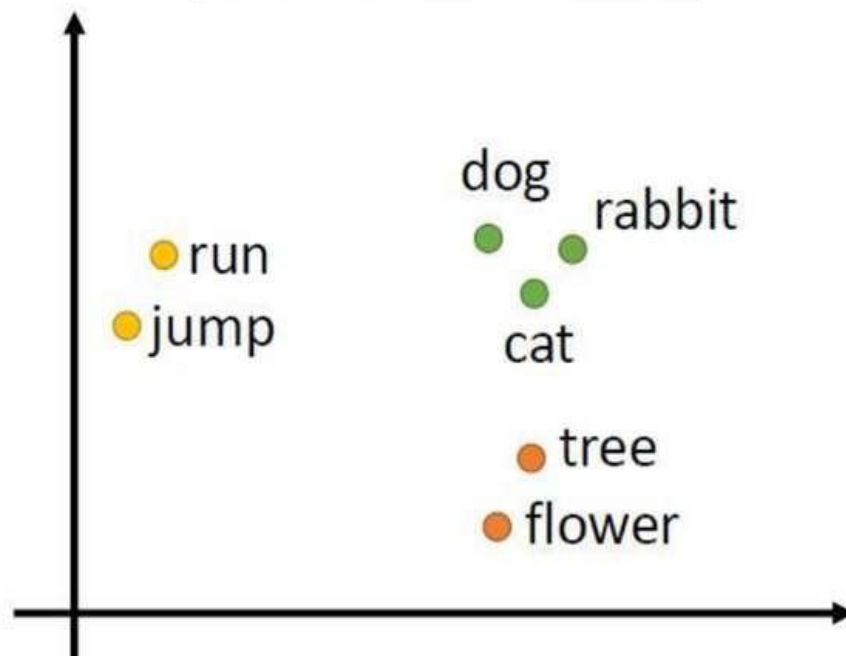
bag = [0 1 0 0 0]

cat = [0 0 1 0 0]

dog = [0 0 0 1 0]

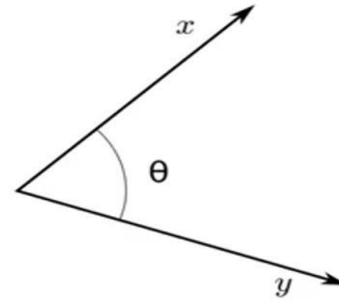
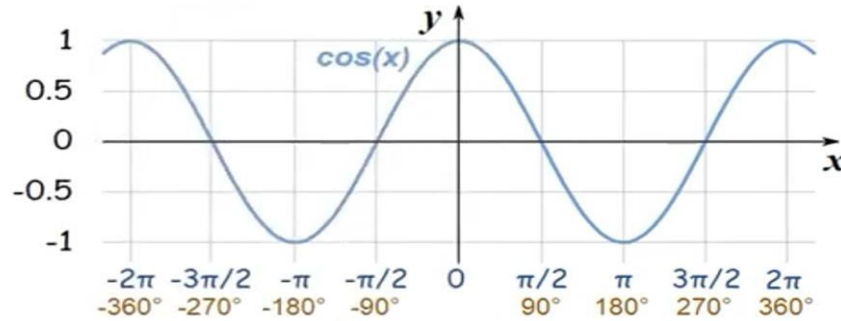
elephant = [0 0 0 0 1]

Word Embedding

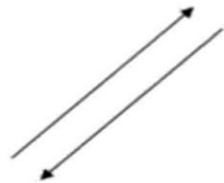


코사인 유사도

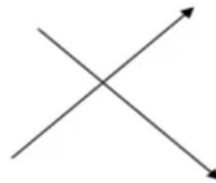
$$-1 \leq \cos(\theta) \leq 1$$



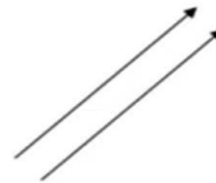
방향에 따른 유사성



유사도-1



유사도0



유사도1

$$\text{Cosine similalrity} = \cos(\theta) = \cos(\mathbf{x}, \mathbf{y}) = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

norm

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

$\langle \mathbf{x}, \mathbf{y} \rangle$: inner product of vectors, $\mathbf{x}$ and $\mathbf{y}$

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{k=1}^n (x_k y_k)$$

$\|\mathbf{x}\|$: the length of vector $\mathbf{x}$

Norm(x)

$$\|\mathbf{x}\| = \sqrt{\sum_{k=1}^n (x_k)^2}$$

코사인 유사도 예시

$$\mathbf{X} = 3 \ 2 \ 0 \ 5 \ 0 \ 0 \ 0 \ 2 \ 0 \ 0$$

$$\mathbf{Y} = 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 2$$

$$\langle \mathbf{X}, \mathbf{Y} \rangle = 3*1 + 2*0 + 0*0 + 5*0 + 0*0 + 0*0 + 0*0 + 2*1 + 0*0 + 0*2 = 5$$

$$\|\mathbf{X}\| = \sqrt{(3*3 + 2*2 + 0*0 + 5*5 + 0*0 + 0*0 + 0*0 + 2*2 + 0*0 + 0*0)} = 6.48$$

$$\|\mathbf{Y}\| = \sqrt{(1*1 + 0*0 + 0*0 + 0*0 + 0*0 + 0*0 + 0*0 + 1*1 + 0*0 + 2*2)} = 2.45$$

$$\cos(\mathbf{X}, \mathbf{Y}) = 0.31$$

11. Chat GPT API



11.1 프롬프트 함수

```
from openai import OpenAI
```

```
client = OpenAI(api_key="your-api-key")
```

```
response = client.chat.completions.create( 아규먼트 )
```

환경변수에 API 키 등록

1. 시작 메뉴 > "환경 변수" 검색 > "시스템 환경 변수 편집" 클릭
2. "환경 변수(N)..." 버튼 클릭
3. "사용자 변수" 또는 "시스템 변수"에서 [새로 만들기] 클릭
 - 변수 이름: `OPENAI_API_KEY`
 - 변수 값: `sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`

환경변수에 API 키 사용

```
import os
import openai
# 환경변수에서 API 키 읽기
api_key = os.getenv("OPENAI_API_KEY")

# openai에 API 키 설정
openai.api_key = api_key

# 예시 요청: GPT 모델에게 질문하기
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo", # 또는 "gpt-4"
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": " hello gpt."}
    ]
)
print(response.choices[0].message["content"])
```

create 함수 아규먼트

아규먼트	데이터 타입	설명
model	str	사용할 모델의 이름. 예: "gpt-3.5-turbo", "gpt-4" 등
messages	list	대화의 메시지 목록. 각 메시지는 딕셔너리 형식으로 { "role": "system/user/assistant", "content": "message" }
temperature	float	샘플링의 창의성 정도를 조절하는 값. 0과 1 사이의 값으로, 0에 가까울수록 더 결정적인 응답을 생성
top_p	float	nucleus sampling의 파라미터. temperature 와 함께 사용하여 출력의 창의성을 조절
n	int	생성할 응답의 수. 기본값은 1입니다. 여러 응답을 생성할 수 있습니다.
stop	str or list	응답 생성 중 멈출 토큰을 지정. 하나 이상의 토큰을 멈추는 기준으로 설정 가능
max_tokens	int	응답에 대해 생성할 최대 토큰 수. 제한을 두어 긴 응답을 방지할 수 있습니다.
presence_penalty	float	모델이 이미 나온 토큰을 다시 생성하는 것을 방지하는 패널티. 기본값은 0이며, 1.0 이상으로 증가시킬 수 있음
frequency_penalty	float	자주 사용된 단어가 다시 등장할 확률을 줄이는 패널티. 기본값은 0
logit_bias	dict	특정 단어에 대한 우선순위를 변경할 수 있는 로그 확률 값. 특정 토큰에 대한 "편향"을 설정
user	str	사용자를 식별할 수 있는 고유 ID, 이 값을 통해 대화에 참여하는 사용자를 추적할 수 있습니다.

model 인자

1. model

- **설명:** `model` 은 생성하려는 응답에 사용할 언어 모델을 지정하는 아규먼트입니다. OpenAI는 여러 종류의 모델을 제공하며, 모델마다 성능, 크기, 특성 등이 다릅니다.
- **주요 모델 예시:**
 - `gpt-3.5-turbo`: GPT-3의 고속 모델로, 비교적 빠르고 적당한 성능을 제공합니다.
 - `gpt-4`: 최신 모델로, GPT-3보다 더 정확하고 다양한 작업을 잘 처리할 수 있습니다.
 - `gpt-4.0-turbo`: GPT-4의 고속 버전으로, 성능은 GPT-4와 비슷하지만 더 빠른 응답을 제공합니다.
- **사용법:** `model` 은 반드시 문자열 형태로 제공되어야 하며, 해당 모델의 이름을 정확하게 입력해야 합니다. 예를 들어:

```
python
```

[📄 코드 복사](#)

```
model="gpt-4"
```

message 인자

2. messages

- **설명:** `messages` 는 모델이 응답을 생성하는 데 필요한 **대화의 맥락**을 제공하는 인자입니다. 이는 대화형 인터페이스에서 모델이 어떤 방식으로 응답할지 결정하는 중요한 요소입니다. `messages` 는 메시지 리스트로, 각 메시지는 ****역할 (role)****과 ****내용 (content)****로 구성됩니다.
- **역할 (role):**
 - `system`: 시스템은 대화의 설정을 제공하는 역할을 합니다. 예를 들어, 모델이 대화에서 어떤 스타일로 응답해야 하는지, 어떤 규칙을 따라야 하는지 등을 정의합니다.
 - `user`: 사용자가 입력한 메시지입니다. 실제로 모델이 응답해야 할 질문이나 요청을 포함합니다.
 - `assistant`: 모델이 생성한 응답 메시지입니다.
- **내용 (content):**
 - 각 메시지에는 해당 메시지의 **내용**이 텍스트 형태로 포함됩니다. 예를 들어, 사용자가 요청한 질문이나 시스템에서 정의한 규칙 등이 포함됩니다.

- **예시:** 대화의 예시는 다음과 같습니다:

python

📄 코드 복사

```
messages = [  
    {"role": "system", "content": "You are a helpful assistant."},  
    {"role": "user", "content": "What is the capital of France?"},  
    {"role": "assistant", "content": "The capital of France is Paris."}  
]
```

여기서:

- `system`: 시스템은 모델에게 "너는 친절한 조수야"라는 역할을 정의합니다.
- `user`: 사용자는 "프랑스의 수도가 뭐야?"라는 질문을 합니다.
- `assistant`: 모델은 이 질문에 대해 "프랑스의 수도는 파리아."라고 답합니다.

functions 인자

OpenAI GPT-4 모델을 사용할 때, **ChatGPT API**에서 **함수 호출 기능**을 사용할 수 있게 하는 아규먼트

- **함수 정의**: functions에는 하나 이상의 함수에 대한 정의를 포함할 수 있습니다. 각 함수는 함수의 이름, 매개변수(입력) 및 함수가 수행하는 작업을 설명하는 정보를 담고 있습니다.
- **모델의 함수 호출**: 모델이 대화 중에 특정 함수가 호출되도록 유도할 수 있습니다. 예를 들어, 사용자가 "날씨를 알려줘"라고 요청하면 모델은 **날씨 API**를 호출하는 함수로 연결할 수 있습니다.

응답(response) 구조1

항목	데이터 타입	설명
id	str	응답의 고유 ID. OpenAI의 API에서 이 응답을 식별하는 데 사용됩니다.
object	str	응답 객체의 타입. 예를 들어, <code>"chat.completion"</code> 과 같은 값을 가질 수 있습니다.
created	int	응답이 생성된 Unix timestamp (초 단위).
model	str	사용된 모델의 이름 (예: <code>"gpt-3.5-turbo"</code>).
choices	list	모델의 응답을 포함하는 리스트입니다. 각 항목은 <code>choice</code> 를 나타내며, 모델이 생성한 다양한 응답을 포함할 수 있습니다.
choices.message	dict	<code>choices</code> 리스트 내 각 항목은 메시지 객체를 포함합니다. 이 객체는 모델의 응답 메시지를 나타내며, <code>role</code> 과 <code>content</code> 를 포함합니다.

응답(response) 구조2

choices.message.role	str	메시지의 역할 (<code>system</code> , <code>user</code> , <code>assistant</code>). 모델이 생성한 응답이 어떤 역할인지 나타냅니다.
choices.message.content	str	모델이 생성한 메시지의 실제 내용입니다. 사용자의 질문에 대한 모델의 응답이 포함됩니다.
usage	dict	API 사용 통계를 제공합니다. 주로 <code>prompt_tokens</code> , <code>completion_tokens</code> , <code>total_tokens</code> 를 포함합니다.
usage.prompt_tokens	int	요청에 포함된 토큰의 수입니다 (입력 텍스트와 관련된 토큰).
usage.completion_tokens	int	생성된 응답에 포함된 토큰의 수입니다 (모델이 생성한 응답에 해당하는 토큰).
usage.total_tokens	int	<code>prompt_tokens</code> 와 <code>completion_tokens</code> 를 합친 총 토큰 수입니다.

Response 예시

```
{
  "id": "chatcmpl-6g5wdvR6j9g2v5zxvnfGpKrK5wV6G",
  "object": "chat.completion",
  "created": 1678650993,
  "model": "gpt-3.5-turbo",
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "Space complexity is a measure of the amount of memory an algorithm needs to run,
                    in relation to the size of the input data."
      },
      "finish_reason": "stop",
      "index": 0
    }
  ],
  "usage": {
    "prompt_tokens": 36,
    "completion_tokens": 25,
    "total_tokens": 61
  }
}
```

11.2 모더레이션모델

- OpenAI에서 제공하는 특정 종류의 AI 모델로, **사용자 입력과 생성된 콘텐츠가 정책에 위배되는지 여부를 검사하는 데 사용**
- 사용자가 요청한 내용이 **위험**하거나 **부적절한** 내용을 포함하고 있는지 감지하고, 그에 맞는 행동을 취할 수 있도록 도와준다

주요 기능

모더레이션 모델의 핵심 목표는 생성된 콘텐츠가 다음과 같은 문제를 일으킬 수 있는지 검사하는 것:

- 유해한 콘텐츠**: 혐오 발언, 폭력적 발언, 차별적인 발언 등.
- 부적절한 언어**: 성적 내용, 음란물, 음해 등의 발언.
- 비윤리적 행동**: 불법적인 활동을 조장하거나 위험한 행동을 유도하는 콘텐츠.
- 스팸**: 부적절하거나 반복적인 메시지, 광고성 메시지.

모더레이션모델 사용

```
from openai import OpenAI  
client = OpenAI(api_key="your-api-key")  
  
response = client.moderations.create(  
    input="This is a very offensive message."  
)  
  
print(response)
```


모더레이션모델 response 예

```
{
  "id": "mod-12345",
  "object": "moderation",
  "model": "text-moderation-001",
  "result": {
    "flagged": true,
    "categories": {
      "hate": true,
      "violence": false,
      "self-harm": false,
      "sexual": false,
      "harassment": false
    },
    "score": 0.98
  }
}
```

- flagged: True이면 콘텐츠가 정책을 위반하여 부적절하다고 판단됩니다.
- categories: 여러 가지 카테고리로 나누어 부적절한 내용이 포함되어 있는지 판단합니다.
- hate: 혐오 발언 여부.
- violence: 폭력적인 내용 여부.
- self-harm: 자해 관련 내용 여부.
- sexual: 성적인 내용 여부.
- harassment: 괴롭힘 관련 내용 여부.
- score: 각 카테고리에 대한 확신도를 나타내는 점수입니다. 일반적으로 0과 1 사이의 값이며, 값이 1에 가까울수록 해당 카테고리에 대한 확신이 높습니다.

11.3 위스퍼

Whisper는 **OpenAI**가 개발한 **음성 인식 모델**입니다. Whisper는 **다양한 언어**로 음성을 텍스트로 변환하는 기능을 제공하며, 주로 **자동 음성 인식 (ASR)** 시스템으로 사용됩니다. 이 모델은 **다양한 음성 데이터를 학습**하여 높은 정확도를 자랑하며, 여러 언어와 방언을 지원합니다.

주요 특징:

1. **다양한 언어 지원**: Whisper는 여러 언어와 방언을 지원하며, 음성 데이터를 텍스트로 정확하게 변환할 수 있습니다. 특히 **다양한 악센트와 발음**을 잘 인식합니다.
2. **음성에서 텍스트로 변환**: 음성을 입력으로 받아 이를 텍스트로 변환하는 **ASR (Automatic Speech Recognition)** 기술을 사용합니다.
3. **강력한 성능**: Whisper는 대규모의 멀티언어 데이터셋에서 학습되어, 다양한 음성 환경에서도 **높은 인식률**을 보입니다.
4. **파일 포맷 지원**: Whisper는 **MP3, WAV, FLAC** 등 여러 음성 파일 포맷을 지원합니다.
5. **오픈소스**: Whisper는 오픈소스로 제공되어, 누구나 사용할 수 있습니다.

11.3 위스퍼

응용 분야:

- **자동 자막 생성:** 동영상 콘텐츠에서 자동으로 자막을 생성하는 데 사용됩니다.
- **음성 비서:** 음성 명령을 텍스트로 변환하여 다양한 작업을 수행하는 시스템에 활용됩니다.
- **회의 기록:** 회의나 인터뷰의 음성을 텍스트로 변환하여 기록하는 데 사용됩니다.
- **다국어 번역:** 여러 언어로 음성을 인식하여 텍스트를 출력하고, 이를 다른 언어로 번역하는 데 활용될 수 있습니다.

```
import openai

openai.api_key = "your-api-key"

# Whisper 모델을 사용하여 음성을 텍스트로 변환
audio_file = open("your-audio-file.mp3", "rb")
response = openai.Audio.transcribe(
    model="whisper-1", # Whisper 모델 사용
    file=audio_file,
    language="en" # 언어 설정 (영어)
)

print(response["text"])
```

11.4 DALL-E

Whisper는 **OpenAI**가 개발한 음성 인식 모델입니다. Whisper는 다양한 언어로 음성을 텍스트로 변환하는 기능을 제공하며, 주로 자동 음성 인식 (**ASR**) 시스템으로 사용됩니다. 이 모델은 다양한 음성 데이터를 학습하여 높은 정확도를 자랑하며, 여러 언어와 방언을 지원합니다.

주요 특징:

1. 다양한 언어 지원: Whisper는 여러 언어와 방언을 지원하며, 음성 데이터를 텍스트로 정확하게 변환할 수 있습니다. 특히 다양한 **악센트와 발음**을 잘 인식합니다.
2. 음성에서 텍스트로 변환: 음성을 입력으로 받아 이를 텍스트로 변환하는 **ASR (Automatic Speech Recognition)** 기술을 사용합니다.
3. 강력한 성능: Whisper는 대규모의 멀티언어 데이터셋에서 학습되어, 다양한 음성 환경에서도 높은 인식률을 보입니다.
4. 파일 포맷 지원: Whisper는 **MP3, WAV, FLAC** 등 여러 음성 파일 포맷을 지원합니다.
5. 오픈소스: Whisper는 오픈소스로 제공되어, 누구나 사용할 수 있습니다.

11.4 DALL-E

주요 사용 사례:

- **디지털 아트 및 디자인:** 디자이너가 창의적인 작업을 할 때 DALL-E를 활용하여 빠르게 아이디어를 시각화하고, 그 후 실제 디자인 작업에 반영할 수 있습니다.
- **게임 디자인:** 게임 개발자들이 DALL-E를 사용해 게임 속 캐릭터, 배경, 아이템 등을 빠르게 시각화하는 데 사용할 수 있습니다.
- **광고 및 마케팅:** 광고 캠페인에서 새로운 이미지나 비주얼을 생성하고, 이를 고객과의 소통에 활용할 수 있습니다.
- **교육 및 연구:** DALL-E는 특정 개념을 시각적으로 표현해야 하는 교육적인 자료 생성에도 유용합니다.

```
import openai
```

```
openai.api_key = "your-api-key"
```

```
response = openai.Image.create(  
    prompt="a futuristic city with flying cars and neon  
lights",  
    n=1, # 생성할 이미지의 개수  
    size="1024x1024" # 이미지 크기  
)
```

```
# 생성된 이미지 URL
```

```
image_url = response['data'][0]['url']  
print(image_url)
```

11.5 챗 GPT가 학습하지 않은 데이터 질문에 답하기

3 가지 방법

1) 파인튜닝

2) 퓨샷러닝: 모델에 전송된 프롬프트에 예시추가

3) RAG: 검색증강생성

11. Chat GPT API



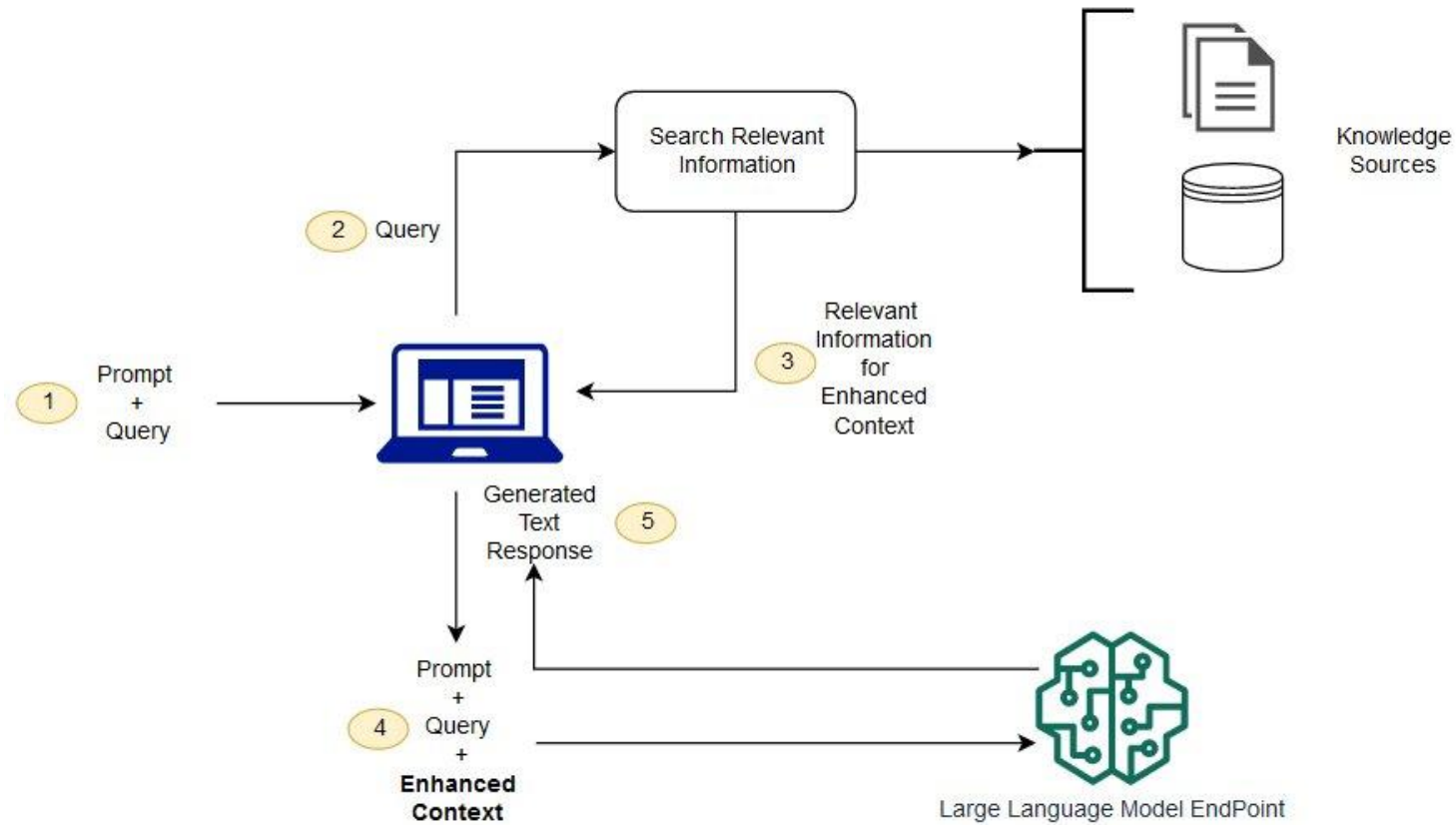
11.6 RAG

RAG(Retrieval-Augmented Generation)는 대규모 언어 모델의 출력을 최적화하여 응답을 생성하기 전에 학습 데이터 소스 외부의 신뢰할 수 있는 지식 베이스(예를 들어 위키피디아)를 참조

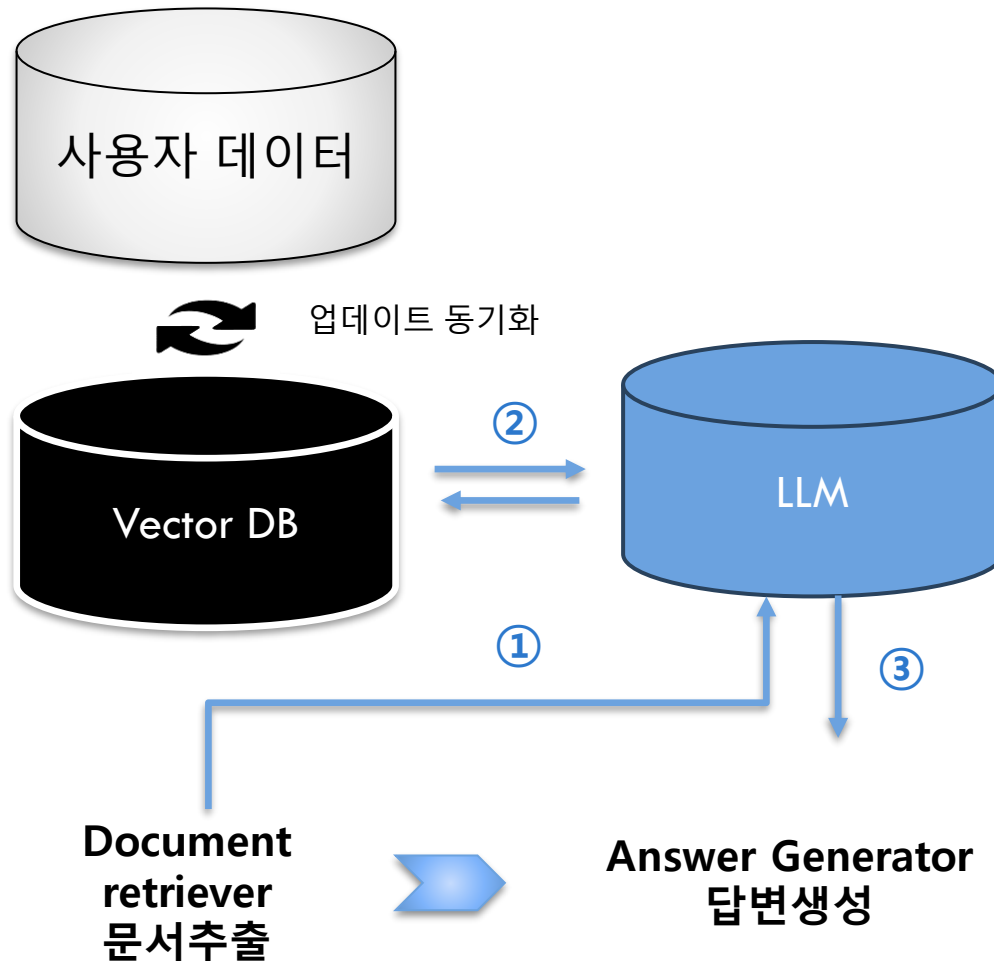
LLM 신뢰성 문제(환각(Hallucination))를 극복하기 위함

할루시네이션 대규모 언어 모델의 출력을 최적화하여 응답을 생성하기 전에 학습 데이터 소스 외부의 신뢰할 수 있는 지식 베이스(예를 들어 위키피디아)를 참조

LLM 신뢰성 문제(환각(Hallucination))를 극복하기 위함



할루시네이션(hallucination)

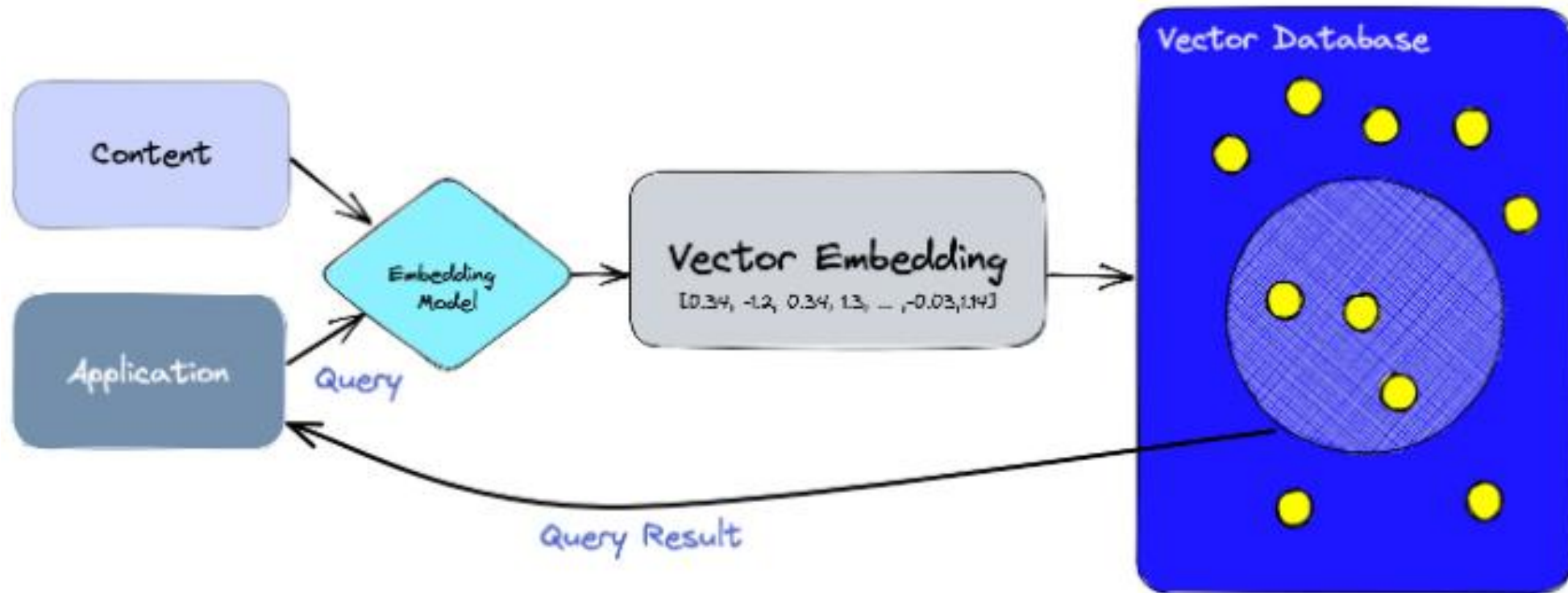


- ① Document retriever from vector DB
- ② Document retriever from vector DB
- ③ 문장완성 답변

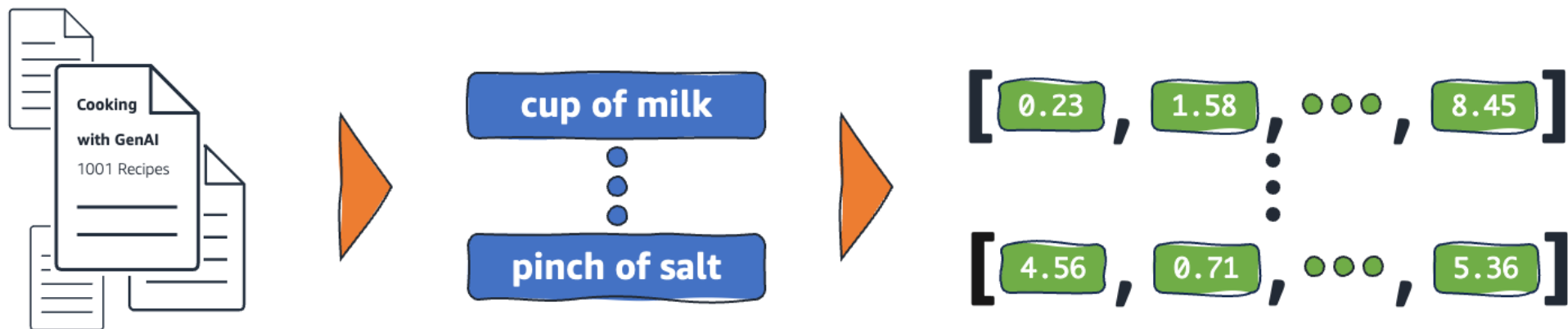
11.7 벡터데이터베이스

- 임베딩된 벡터데이터 저장소
- 데이터 관리 및 검색 기능도 제공
- 데이터의 정확한 한 값이 아닌 ‘유사성’을 바탕으로 검색

벡터 데이터베이스



임베딩



유사도 검색(검색: 고객나이가 30이상인 것은)

- 일반 데이터베이스

Select * from customer where age>=30

- 벡터 데이터베이스

1) POST /search: 벡터 데이터베이스(Vector Database)에서 검색을 수행하는 API 요청

* 벡터 데이터베이스는 POST 요청을 통해 검색을 수행

2) query_vector: 사용자의 질문을 임베딩(Embedding)한 벡터 표현

3) Top_k: 반환받고자 하는 가장 유사한 문서의 수

POST /search

```
{
    query_vector:[ 0.13, -0.24, ..., .0.78]
    top_k: 5
}
```

유사도 검색

구분	일반 데이터베이스(SQL)	벡터 데이터베이스
검색 방식	정확한 조건 기반	유사도 기반 (Nearest Neighbor Search)
예제	<code>age >= 30</code> 조건	30세 이상과 유사한 개념 검색
결과	조건을 만족하는 모든 행 반환	가장 가까운 top_k개의 벡터 반환
사용 사례	고객 나이 필터링, 정확한 검색	AI 추천 시스템, 유사한 고객 찾기

벡터 데이터베이스의 검색 방식

벡터 간의 유사도를 계산하는 대표적인 방법:

- 코사인 유사도(Cosine Similarity)
- 유클리드 거리(Euclidean Distance)
- 내적(Dot Product)

벡터 데이터베이스에서는 '30세 이상'을 정확한 숫자로 필터링하는 것이 아니라, 30세 이상 고객과 개념적으로 유사한 데이터를 찾는 것

벡터데이터베이스 종류

- ▶ 파인콘(pinecone)
- ▶ 밀버스(milvus)
- ▶ 쿼드런트(qdrant)
- ▶ 크로마(chroma)
- ▶ 엘라스틱서치(elasticsearch)
- ▶ 파이스(faiss): 인덱스를 이용한 빠른 검색

등

벡터데이터베이스 종류 및 특징

벡터 DB	특징	장점	단점
FAISS (Facebook AI Similarity Search)	- Meta(Facebook)에서 개발 - 대규모 벡터 데이터 검색 최적화 - GPU 가속 지원	- 속도가 빠름 (HNSW, IVF 등 최적화된 인덱싱 사용) - 대용량 데이터 처리에 강함	- 분산 지원 부족 - 실시간 삽입/삭제 어려움
Milvus	- 오픈소스 벡터 데이터베이스 - 분산 시스템 지원 (Cloud, Kubernetes 가능) - 다양한 인덱스 알고리즘 제공	- 실시간 벡터 삽입/삭제 가능 - 확장성이 뛰어나 (클러스터 구성 가능)	- 단순 설치 환경에서는 설정이 복잡할 수 있음
Pinecone	- 클라우드 기반 벡터 검색 서비스 - 관리형 DB로 운영 부담 없음	- 인프라 관리 불필요 (SaaS 방식) - 빠른 응답 속도	- 온프레미스(On-Premise) 지원 부족 - 비용이 상대적으로 높을 수 있음
Weaviate	- 오픈소스 벡터 DB - GraphQL 지원 (텍스트 검색 기능 강화)	- 텍스트 & 벡터 혼합 검색 가능 - 플러그인 확장성 높음	- 성능이 FAISS나 Milvus 대비 다소 떨어질 수 있음

Annoy (Approximate Nearest Neighbors Oh Yeah)	- Spotify에서 개발 - 디스크 기반 인덱스 사용 (메모리 절약)	- 가볍고 빠름 - 설치 및 사용이 간편	- 실시간 데이터 업데이트 불가능 - 대규모 데이터 처리에는 비효율적
Vespa	- 벡터 검색 + 정형 데이터 검색 가능 - ML 기반 랭킹 조합 가능	- 벡터 검색과 일반 SQL-like 검색 모두 지원 - 엔터프라이즈 환경 적합	- 설정 및 운영이 복잡할 수 있음
Qdrant	- Rust 기반 고성능 벡터 DB - 실시간 삽입/삭제 지원	- 속도가 빠르고 가볍음 - 클러스터 지원 (확장성)	- 커뮤니티가 다른 DB보다 작음

3-1. 퓨샷 프롬프트 (Few-Shot) 예

✅ 예시: 언어 감지

💬 프롬프트:

makefile

📄 복사

✎ 편집

다음 문장이 어떤 언어인지 감지해줘.

예시:

문장: "Bonjour tout le monde"

언어: 프랑스어

문장: "Guten Tag, wie geht's?"

언어: 독일어

문장: "こんにちは、元気ですか？"

언어:

apple: fruit

orange: fruit

zucchini: vegetable

tomato:

🔍 설명:

- 모델은 앞의 예시를 보고 패턴을 학습
- 유사한 방식으로 마지막 문장의 언어를 추론함

12. 랭체인



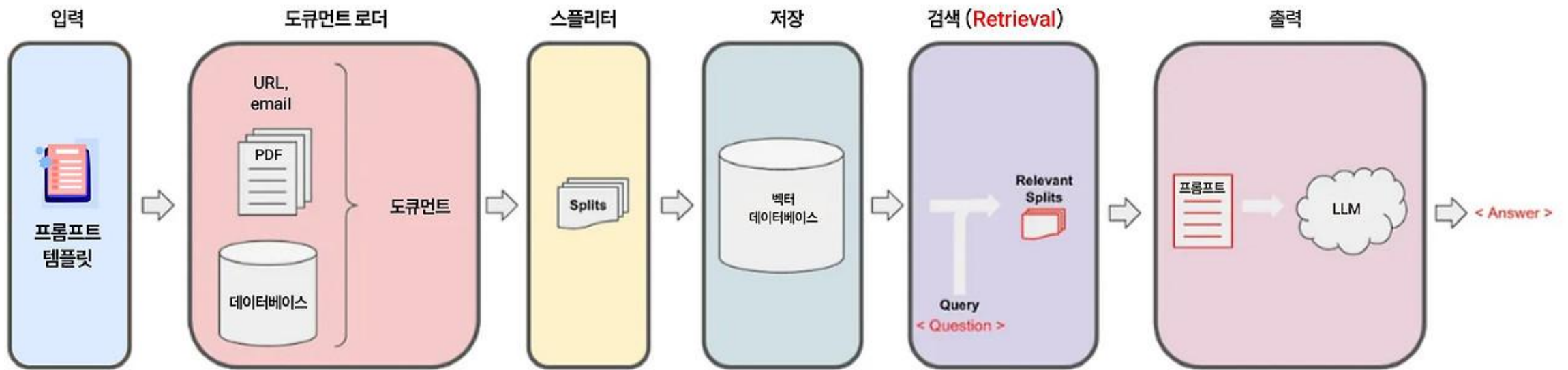
LangChain

랭체인을 사용하는 이유

◆ LangChain을 사용하는 이유

이유	설명
1. LLM을 쉽게 연결	OpenAI, Hugging Face, Cohere 등 다양한 LLM API를 손쉽게 호출 가능
2. 프롬프트 관리 용이	<code>PromptTemplate</code> 을 사용해 체계적으로 프롬프트 설계 및 재사용 가능
3. 메모리 활용 가능	대화 상태를 저장해 대화형 챗봇 개발에 유리
4. 벡터 데이터베이스와 통합 가능	FAISS, Pinecone, Weaviate 등과 쉽게 연결하여 장기 기억 구현 가능
5. 다양한 데이터 소스 활용	PDF, 웹페이지, 데이터베이스 등에서 정보를 읽어와 모델 입력으로 활용
6. 체인(Chain) 기반 워크플로우	LLM 응답을 조합하여 여러 단계의 논리적인 처리가 가능
7. 에이전트 기반 동적 실행	도구(Tool)과 연동해 LLM이 동적으로 판단하고 실행할 수 있도록 지원

랭체인 구성요소



LangChain

랭체인 구성도

